VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

FACULTY OF COMPUTER SCIENCE AND ENGINEERING

REPORT

CAPSTONE PROJECT

SEMESTER 252 ACADEMIC YEAR 2025-2026

DOMAIN GENERALIZATION FOR ADVERSARIAL ROBUSTNESS

MAJOR: COMPUTER SCIENCE

COUNCIL:

SUPERVISOR(s): Nguyễn An Khương

Văn Minh Hào

REVIEWER:

——o0o——

STUDENT 1: Nguyễn Anh Kiệt - 2252403

STUDENT 2: Lê Văn Đức Anh - 2252027

STUDENT 3: Nguyễn Tiến Thành - 2053437

HO CHI MINH CITY, May 2026

Instructor’s Signature

_____ Date: _____

NGUYỄN AN KHƯƠNG (Instructor)

...

Faculty of Computer Science and Engineering

Declaration of Authenticity

We declare that we solely conducted this specialized project, under the supervision of Dr. Nguyen An Khuong at the Faculty of Computer Science and Engineering, Vietnam National University - Ho Chi Minh City University of Technology and Mr. Van Minh Hao at University of Arkansas.

We have taken care to properly acknowledge and document all external sources and references used in the project.

If there is any instance of plagiarism, we are ready to accept the consequences. Ho Chi Minh City University of Technology - Vietnam National University HCMC will not be held responsible for any copyright violations that may have occurred during my research.

Ho Chi Minh City, December 2025

Authors

Acknowledgement

We would like to express my appreciation to Dr. Nguyen An Khuong for his invaluable guidance. Our research has greatly benefited from his deep knowledge, perceptive criticism, and constant support.

Besides, we are grateful for the help from Mr. Van Minh Hao for valuable comments for research aspect and our proposed pipeline, including pointing out strengths as well as weaknesses of previous works on adversarial attack so that we can elaborate on our ideas more effectively.

In addition, we would like to extend our gratitude to our family. Their unwavering faith in our abilities and constant encouragement have been my pillars of strength. Their belief in our potential has been a constant source of motivation and resilience. We are forever grateful for their love and support.

Abstract

This thesis addresses the critical vulnerability of deep learning models to adversarial perturbations, particularly the limitation where standard defense mechanisms fail to generalize across different threat models. Our research proposes a novel perspective that treats distinct families of adversarial attacks as separate ”domains,” aiming to identify and quantify the ”cross-attack robustness gap” inherent in current training paradigms.

We establish a comprehensive experimental framework to evaluate the transferability of robustness across diverse adversarial environments. By subjecting models trained on specific perturbation norms to a wide spectrum of unseen attack algorithms, the study systematically measures the performance degradation that occurs when shifting between these domains.

Ultimately, this quantitative analysis serves as the empirical foundation for the subsequent phase of research. By characterizing the domain shifts between attack families, we pave the way for integrating Domain Generalization principles into adversarial training. This progression aims to directly mitigate the observed robustness gaps, ensuring that future models can effectively generalize to unforeseen and heterogeneous adversarial scenarios.

Contents

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Definitions
AA	AutoAttack
AT	Adversarial Training
CW	Carlini–Wagner
DG	Domain Generalization
DNN	Deep Neural Network
FGSM	Fast Gradient Sign Method
IRM	Invariant Risk Minimization
PGD	Projected Gradient Descent
REx	Risk Extrapolation

Chapter 1

Introduction

1.1 Adversarial Robustness and the Cross-Attack Gap

1.2 Stage 1 Findings and Their Limitations

1.3 Problem Statement

1.4 Research Questions and Hypotheses

1.5 Contributions of This Work

1.6 Scope and Limitations

1.7 Report Organization

Chapter 2

Background

2.1 Notation and Learning Setting

This section introduces the basic notation and learning setting used throughout the report. Since Stage 1 already established the general background on deep neural networks, adversarial examples, adversarial attacks, and adversarial training, the present section provides a concise recap of these foundations before extending them toward the Stage 2 methodology. In particular, we focus on the concepts that are necessary for formulating adversarial training under multiple attack-induced domains.

We consider a supervised classification problem with input space $\mathcal{X} \subseteq \mathbb{R}^d$ and label space $\mathcal{Y} = \{1, \dots, K\}$. A training dataset is denoted by

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n},$$

where each sample (x_i, y_i) is drawn from an underlying data distribution P_{XY}. The goal is to learn a classifier that predicts the correct label for unseen inputs while remaining stable under adversarial perturbations. This stability requirement is central to adversarial robustness and becomes the main motivation for the multi-attack and group-aware training methods studied in later chapters.

2.1.1 Deep Neural Networks and Supervised Classification

Deep neural networks (DNNs) have become the dominant paradigm for modern classification tasks due to their ability to learn hierarchical feature representations directly from data. A DNN consists of multiple layers of nonlinear transformations, where early lay-

ers often capture low-level patterns such as edges or textures, while deeper layers encode more abstract and task-specific semantic features.

Formally, a neural network with parameters θ defines a function

$$f_\theta : \mathbb{R}^d \to \mathbb{R}^K,$$

which maps an input $x \in \mathbb{R}^d$ to a vector of K real-valued outputs. These outputs are commonly referred to as logits and represent the modelâ€™s unnormalized scores for each class. The predicted class is obtained by selecting the class with the largest logit:

$$\hat{y} = \arg\max_{k \in \{1,\dots,K\}} f_\theta(x)_k.$$

For probabilistic interpretation, the logits are usually passed through the softmax function

$$p_\theta(y = k \mid x) = \frac{\exp(f_\theta(x)_k)}{\sum_{j=1}^{K} \exp(f_\theta(x)_j)}.$$

2.1.2 Loss Functions and Gradient-Based Optimization

The model is trained by minimizing the cross-entropy loss between the predicted distribution and the ground-truth label. For a single training sample (x, y), the loss is defined as

$$\ell(f_\theta(x), y) = -\log p_\theta(y \mid x).$$

Standard empirical risk minimization trains the model by minimizing the average loss over the training set

$$\min_\theta \frac{1}{n} \sum_{i=1}^{n} \ell(f_\theta(x_i), y_i).$$

Although this objective is effective for improving clean classification accuracy, it does not explicitly require the classifier to be stable under small input perturbations. In high-dimensional input spaces, a perturbation δ with small norm can still significantly change the logits and alter the predicted label:

$$\arg\max_k f_\theta(x)_k \neq \arg\max_k f_\theta(x+\delta)_k, \quad \|\delta\|_p \leq \varepsilon.$$

This phenomenon motivates the study of adversarial robustness. In Stage 1, this vulnerability was analyzed through a diverse suite of attacks, including FGSM, PGD, Carlini–Wagner, DeepFool, and AutoAttack. The results showed that robustness obtained against one attack configuration does not necessarily transfer to other attacks, revealing a cross-attack robustness gap. Stage 2 builds on this observation by treating different attack mechanisms as distinct domains and by developing group-aware training objectives to reduce this gap.

2.2 Adversarial Examples and Threat Models

Adversarial examples are inputs that have been intentionally perturbed to cause a machine learning model to make incorrect predictions, while the perturbations remain small enough to preserve the original semantic label. Given a clean input $x \in \mathbb{R}^d$ with true label y, an adversarial example can be written as

$$x^{adv} = x + \delta,$$

where δ denotes the adversarial perturbation. The objective of the attacker is to find a perturbation that changes the model prediction while remaining within a constrained perturbation set. In the untargeted setting, the attacker aims to produce x^{adv} such that

$$\arg\max_k f_\theta(x^{adv})_k \neq y.$$

In contrast, a targeted attack attempts to force the model to predict a specific target class $y_{target} \neq y$:

$$\arg\max_k f_\theta(x^{adv})_k = y_{target}.$$

2.2.1 Lp-Norm Perturbation Sets

Most adversarial attacks constrain the perturbation by an ℓ_p-norm budget. The feasible perturbation set is commonly defined as

$$\mathcal{S}_p(\varepsilon) = \{\delta \in \mathbb{R}^d : \|\delta\|_p \leq \varepsilon\},$$

where $p \in \{2, \infty\}$ is the norm type and $\varepsilon > 0$ controls the maximum allowed perturbation size. The adversarial example is also clipped to the valid input range, typically $x^{adv} \in [0,1]^d$ for normalized images. Therefore, the adversary searches within the feasible set

$$x^{adv} \in (x + \mathcal{S}_p(\varepsilon)) \cap [0,1]^d.$$

Under this formulation, adversarial example generation can be viewed as an inner maximization problem. For a given model f_θ, loss function ℓ, and clean sample (x, y), the attacker seeks

$$\delta^* = \arg\max_{\delta \in \mathcal{S}_p(\varepsilon)} \ell(f_\theta(x + \delta), y).$$

The resulting adversarial example $x + \delta^*$ corresponds to an input within the allowed perturbation region that maximizes the modelâ€™s loss. In practice, this maximization is approximated by different attack algorithms, such as FGSM, PGD, Carlini–Wagner, DeepFool, or AutoAttack. Stage 1 used these attacks as a diverse evaluation suite to measure how robust a PGD-trained model remains when exposed to attacks beyond the one used during training.

A central assumption in adversarial robustness is label preservation. Although the input is perturbed, the true semantic class is assumed to remain unchanged:

$$y(x) = y(x + \delta), \quad \text{for } \|\delta\|_p \leq \varepsilon.$$

This assumption distinguishes adversarial perturbations from natural distribution shifts or semantic transformations that genuinely change the class. For example, in image classification, a small imperceptible perturbation to a CIFAR-10 image should not change a “cat” into a “dog” from a human perspective. If the model misclassifies the perturbed image, the error reflects a lack of robustness rather than ambiguity in the data.

Different attacks produce perturbations with different geometric and optimization characteristics. Gradient-based attacks such as FGSM and PGD directly use the gradient of the loss with respect to the input. Optimization-based methods such as Carlini–Wagner

search for small perturbations that satisfy a misclassification objective. Boundary-based methods such as DeepFool estimate the closest decision boundary, while ensemble methods such as AutoAttack combine multiple attack strategies to provide a stronger and more reliable robustness evaluation. These differences are important because a model that performs well against one attack may still fail against another.

This observation is the key motivation for the Stage 2 formulation. Instead of treating attacks only as evaluation tools, we interpret each attack mechanism as inducing a distinct adversarial domain. For an attack e, let $\mathcal{A}_e$ denote the corresponding adversarial example generator. The adversarial sample generated by this attack can be written as

$$x_e^{adv} = \mathcal{A}_e(x, y, f_\theta).$$

The risk associated with attack domain e is then defined as

$$R_e(\theta) = \mathbb{E}_{(x,y)\sim P_{XY}} \left[\ell(f_\theta(x_e^{adv}), y) \right].$$

From this perspective, the goal is not only to minimize adversarial loss for a single attack, but also to control the variation of risks across multiple attack domains. A model trained only on one attack may achieve low $R_e(\theta)$ for the training attack but still suffer high risk under another attack e'. This mismatch gives rise to the cross-attack robustness gap, which Stage 1 empirically identified and Stage 2 aims to reduce using multi-attack and group-aware optimization methods.

2.2.2 Whitebox, Graybox, and Blackbox Access

In this project, we consider adversarial perturbations constrained by an ℓ_p-norm budget, as introduced in the previous section. For a clean input x, the adversary searches for an adversarial example $x^{adv} = x + \delta$ such that

$$\|\delta\|_p \leq \varepsilon, \quad x^{adv} \in [0, 1]^d,$$

where $p \in \{2, \infty\}$ and ε controls the perturbation magnitude. In addition to the perturbation constraint, the adversaryâ€™s access to the model determines the type of threat model. We distinguish three common settings: white-box, gray-box, and black-box attacks.

White-box threat model In the white-box setting, the adversary has full access to the model architecture, parameters, loss function, and gradients. This is the strongest and most commonly used setting for evaluating first-order robustness. Given full access to f_θ, the attacker can compute gradients of the loss with respect to the input:

$$\nabla_x \ell(f_\theta(x), y),$$

and use them to construct adversarial examples. Attacks such as FGSM and PGD are typical white-box attacks because they directly rely on input gradients to increase the classification loss.

The white-box setting is particularly important for adversarial training. During training, the model itself is used to generate adversarial examples through an inner maximization procedure. For example, PGD-based adversarial training approximates

$$\max_{\delta \in \mathcal{S}_p(\varepsilon)} \ell(f_\theta(x+\delta), y)$$

using multiple projected gradient ascent steps. Since the attack is generated using the current modelâ€™s gradients, adversarial training naturally operates in a white-box setting.

In Stage 2, the main training attacks are also considered under the white-box threat model. This allows the training procedure to generate strong adversarial samples from the current model and to optimize against them. However, as observed in Stage 1, robustness under a white-box training attack does not necessarily imply robustness against other white-box attacks with different optimization strategies, norms, or loss functions.

Gray-box threat model The gray-box setting represents an intermediate level of adversarial knowledge. In this case, the adversary may not have full access to the target modelâ€™s parameters or gradients, but may have access to related information such as the model architecture, training procedure, dataset, or a surrogate model. Adversarial examples are often generated on a surrogate model and then transferred to the target model.

Formally, let f_{θ_s} be a surrogate model and f_{θ_t} be the target model. A gray-box adversarial example can be generated as

$$x^{adv} = \mathcal{A}(x, y, f_{\theta_s}),$$

and then evaluated on the target model f_{θ_t}. The attack succeeds if

$$\arg\max_k f_{\theta_t}(x^{adv})_k \neq y.$$

This setting is important because adversarial examples often transfer across models. A perturbation generated to fool one model can also fool another model, especially when the two models share similar architectures, training data, or learned decision boundaries.

In this project, gray-box evaluation is useful for measuring transfer robustness. While white-box evaluation tests whether a model is robust to attacks generated directly against itself, gray-box evaluation tests whether robustness generalizes across source and target models. This is especially relevant to the attacks-as-domains perspective, because transfer attacks can be interpreted as another form of distribution shift: the perturbation is generated from one modelâ€™s loss landscape but evaluated on another modelâ€™s decision boundary.

Black-box threat model In the black-box setting, the adversary does not have access to the model parameters or gradients. Instead, the attacker can only query the model and observe limited outputs, such as predicted labels, confidence scores, or logits. Depending on the available output information, black-box attacks can be further divided into score-based and decision-based attacks.

In score-based attacks, the adversary can observe confidence scores or logits and use them to guide the search for adversarial examples. Square Attack is an example of a score-based black-box attack. It performs random localized perturbation updates and accepts changes that improve the attack objective, without requiring gradient information.

In decision-based attacks, the adversary only observes the final predicted label. This is an even more restrictive setting because the attacker cannot directly measure how close an input is to the decision boundary using scores or probabilities. Such attacks typically require more queries and are not the primary focus of this report.

Black-box evaluation is important because it helps detect whether a defense relies on misleading or uninformative gradients. Some defenses may appear robust against gradient-based white-box attacks but fail under gradient-free black-box attacks. This phenomenon

is often associated with gradient masking, where gradients become difficult to use without the model becoming genuinely robust. Therefore, including black-box or score-based components in the evaluation suite provides a stronger robustness assessment.

2.3 Adversarial Attack Algorithms

2.3.1 Fast Gradient Sign Method with Random Start (FGSM-RS)

2.3.2 Projected Gradient Descent (PGD)

2.3.3 Carlini–Wagner Attack (CW)

2.3.4 DeepFool

2.3.5 Decoupled Direction and Norm (DDN)

2.3.6 Momentum Iterative Fast Gradient Sign Method (MI-FGSM)

2.3.7 Targeted Projected Gradient Descent (TPGD)

2.3.8 AutoAttack

2.4 Adversarial Training as Min-Max Optimization

Single-attack adversarial training.

Multi-attack adversarial training.

Trade-off intuition.

2.5 Distributionally Robust Optimization

2.5.1 From ERM to Worst-Group Risk

2.5.2 Group DRO Formulation

Exponentiated-gradient updates.

Regularization in overparameterized models.

DRO in adversarial training.

2.6 Domain Generalization and Clustering

2.6.1 Domain Generalization: Learning Beyond Known Distributions

2.6.2 K-Means Clustering for Group Discovery

2.7 Statistical Tools for Multi-Seed Evaluation

2.7.1 Paired Tests and Bootstrap Confidence Intervals

2.7.2 Effect Size and Multiple-Comparison Correction

Repeated random-seed evaluation.

Chapter 3

Related Work

3.1 Multi-Attack Adversarial Training

3.2 Group DRO Applied to Robustness

3.3 Domain Generalization and Adversarial Robustness

3.4 Cluster Discovery in Robust Training Pipelines

3.5 The Remaining Gap and Motivation for This Work

Chapter 4

Proposed Methodology

4.1 Methodology Overview

Stage 1 showed that adversarial robustness does not transfer uniformly across attack families. A model trained against one threat model can remain vulnerable to attacks with different perturbation geometries, optimization procedures, or loss objectives. This motivates the Stage 2 methodology: instead of treating all adversarial examples as samples from one homogeneous distribution, this chapter models attack-induced examples as domains and asks how training can emphasize weak domains or latent hard groups without losing the stability of the uniform multi-attack baseline.

The chapter develops this progression step by step. Section 4.2 first formulates multi-attack adversarial training under the attacks-as-domains perspective. Section 4.3 then defines Multi-AT, the uniform multi-attack baseline used alongside the single-attack PGD-AT and DDN-AT baselines. Section 4.4 introduces AttackDRO, which applies Group DRO over source attack identities, but this still assumes that each attack forms one homogeneous group.

The remaining sections address that limitation. Section 4.5 introduces AttackDRO++, which replaces attack identities with discovered clusters. Section 4.6 augments those clusters with gradient fingerprints so that grouping can reflect optimization-level difficulty, not only representation similarity. Finally, Section 4.7 adds a uniform anchor that keeps the final AttackDRO++ objective tied to the Multi-AT training signal while still allowing cluster-level correction.

4.2 Multi-Attack Training as a Domain Problem

4.2.1 Setup and Notation

Let

$$\mathcal{D}_{\text{train}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n} \tag{4.1}$$

denote the supervised training dataset, where $\mathbf{x}_i \in \mathcal{X}$ is an input image and $y_i \in \{1, \dots, C\}$ is its class label. Let f_θ be a classifier parameterized by θ, and let $\ell(f_\theta(\mathbf{x}), y)$ denote the cross-entropy loss.

We denote the logits produced by the model as

$$\mathbf{z} = f_\theta(\mathbf{x}) \in \mathbb{R}^C,$$

and the penultimate-layer representation as

$$\mathbf{h} = \phi_\theta(\mathbf{x}) \in \mathbb{R}^{d_h}.$$

The representation $\phi_\theta(\cdot)$ is later used for cluster discovery.

We define a set of source attack domains

$$\mathcal{A}_{\text{src}} = \{A_1, A_2, \dots, A_M\}, \tag{4.2}$$

where each attack operator A_m is characterized by a threat model, perturbation budget, optimization procedure, number of inner steps, and loss objective. Given a clean sample $(\mathbf{x}_i, y_i)$, the adversarial example produced by source attack A_m is

$$\mathbf{x}_i^{(m)} = A_m(f_\theta, \mathbf{x}_i, y_i). \tag{4.3}$$

In the main experiments, we use two source attacks: a PGD-ℓ_∞ attack and a DDN-ℓ_2 attack. This choice exposes the model to two common adversarial norm families during training. Additional attacks are reserved as heldout evaluation attacks and are never used for optimization.

4.2.2 Multi-Attack Risk and Its Limitations

The empirical adversarial risk under source attack A_m is

$$R_m(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell\Big(f_\theta\Big(\mathbf{x}_i^{(m)}\Big), y_i\Big). \tag{4.4}$$

A standard multi-attack objective minimizes the average risk across source attacks:

$$\min_\theta \frac{1}{M}\sum_{m=1}^{M} R_m(\theta). \tag{4.5}$$

This objective improves attack diversity compared with single-attack adversarial training. However, it treats all source attacks and all adversarial examples with equal importance. This creates two limitations.

First, if one source attack consistently induces higher loss than another, equal weighting may underemphasize the harder attack. Second, adversarial examples within the same attack are not homogeneous. Two samples generated by the same algorithm can differ substantially in classification margin, representation, and optimization direction. These within-attack differences are not captured by Equation (4.5).

The central goal of this chapter is therefore to construct adaptive objectives that can identify and upweight hard groups while preserving the stability of the uniform multi-attack training signal.

4.3 Uniform Multi-Attack ERM Baseline

Uniform Multi-Attack ERM serves as the main baseline for this work. It trains on multiple source attacks within the same minibatch but assigns equal weight to all generated adversarial examples. This makes it a strong and simple reference point for evaluating whether adaptive reweighting provides additional benefit.

Given a minibatch

$$\mathcal{B} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N},$$

we partition it into M disjoint subsets

$$\mathcal{B}_1, \dots, \mathcal{B}_M, \qquad \mathcal{B}_m \cap \mathcal{B}_{m'} = \emptyset \ \text{ for } m \neq m', \qquad \bigcup_{m=1}^{M} \mathcal{B}_m = \mathcal{B}.$$

Each subset $\mathcal{B}_m$ is assigned to one source attack A_m. For each $(\mathbf{x}_i, y_i) \in \mathcal{B}_m$, we generate

$$\mathbf{x}_i^{\text{adv}} = A_m(f_\theta, \mathbf{x}_i, y_i).$$

The adversarial examples from all source attacks are then concatenated into one mixed adversarial batch. The uniform multi-attack loss is

$$\mathcal{L}_{\text{ERM}} = \frac{1}{N} \sum_{i=1}^{N} \ell(f_\theta(\mathbf{x}_i^{\text{adv}}), y_i)\,. \tag{4.6}$$

In our main setting, the source attacks are PGD-ℓ_∞ and DDN-ℓ_2, so the model is exposed to both ℓ_∞ and ℓ_2 perturbation geometries during training. However, the objective in Equation (4.6) treats all adversarial examples equally. It does not explicitly identify which attack, class, or latent subgroup is harder. This limitation motivates the adaptive reweighting methods introduced in the following sections.

4.4 AttackDRO: Group DRO Over Attack Identities

AttackDRO extends Uniform Multi-Attack ERM by applying Group Distributionally Robust Optimization (GroupDRO) over attack identities. Each source attack A_m is treated as one group. The method maintains an attack-group weight vector

$$\mathbf{q} = (q_1, \dots, q_M) \in \Delta^M, \tag{4.7}$$

where Δ^M is the M-dimensional probability simplex.

For each attack group m, the group loss is

$$\mathcal{L}_m = \frac{1}{|\mathcal{B}_m|} \sum_{i \in \mathcal{B}_m} \ell(f_\theta(\mathbf{x}_i^{\text{adv}}), y_i)\,. \tag{4.8}$$

The AttackDRO objective is then

$$\mathcal{L}_{\text{AttackDRO}} = \sum_{m=1}^{M} q_m \mathcal{L}_m. \tag{4.9}$$

The group weights are updated using an exponentiated-gradient (EG) rule. Let s_m be the difficulty score of attack group m. The update is

$$q_m \leftarrow q_m \exp(\eta_q s_m), \qquad m = 1, \dots, M, \tag{4.10}$$

followed by normalization and floor projection:

$$q_m \geq q_{\min}, \qquad \sum_{m=1}^{M} q_m = 1. \tag{4.11}$$

The floor constraint prevents any attack group from being completely ignored.

In our formulation, the difficulty score is based on the classification margin. For a sample i with logits $\mathbf{z}_i$, define the margin as

$$r_i = z_{i,y_i} - \max_{c \neq y_i} z_{i,c}. \tag{4.12}$$

A large positive margin indicates a confident correct prediction, while a small or negative margin indicates a hard or misclassified sample. We convert this margin into a per-sample difficulty score using

$$s_i = \text{softplus}(-r_i/\tau) = \log(1 + \exp(-r_i/\tau)), \tag{4.13}$$

where $\tau > 0$ is a temperature parameter. The group-level difficulty score is the average difficulty within group m:

$$s_m = \frac{1}{|\mathcal{B}_m|} \sum_{i \in \mathcal{B}_m} s_i. \tag{4.14}$$

AttackDRO can adaptively emphasize whichever source attack is currently harder. However, it relies on a coarse grouping assumption: all examples generated by the same attack are treated as one homogeneous group. In practice, two PGD-ℓ_∞ examples may have very different margins, representations, and gradient directions. Conversely, examples generated by different attacks may share similar failure modes. Thus, the EG update can only adapt at the attack level and may miss finer-grained difficulty structure within and across attacks. The cluster-based extension in the next section addresses this limitation.

4.5 AttackDRO++: Group DRO Over Discovered Clusters

AttackDRO++ replaces the manually defined attack groups in AttackDRO with latent clusters discovered from adversarial examples. The central hypothesis is that attack identity is too coarse as a grouping criterion. Two adversarial examples generated by different attacks may share the same failure mode, while two examples generated by the same attack may have very different margins, representations, and optimization direc-

tions. Cluster-based grouping allows the DRO objective to adapt to hard subgroups that may cut across attack boundaries.

For each adversarial example $\mathbf{x}_i^{\text{adv}}$, the model extracts a representation vector

$$\mathbf{h}_i = \phi_\theta(\mathbf{x}_i^{\text{adv}}), \tag{4.15}$$

and a clustering model assigns the example to a latent group

$$c_i \in \{1, \dots, K\}. \tag{4.16}$$

Let $\mathcal{B}_k$ denote the subset of the current minibatch whose adversarial examples are assigned to cluster k:

$$\mathcal{B}_k = \{i \in \{1, \dots, N\} : c_i = k\}. \tag{4.17}$$

The cluster loss is

$$\mathcal{L}_k = \frac{1}{|\mathcal{B}_k|} \sum_{i \in \mathcal{B}_k} \ell\big(f_\theta(\mathbf{x}_i^{\text{adv}}), y_i\big)\,. \tag{4.18}$$

AttackDRO++ maintains a cluster-level weight vector

$$\mathbf{q} = (q_1, \dots, q_K) \in \Delta^K. \tag{4.19}$$

Because not every cluster appears in every minibatch, the cluster weights are renormalized over the clusters present in the current minibatch. Let

$$\mathcal{C}_{\mathcal{B}} = \{k \in \{1, \dots, K\} : |\mathcal{B}_k| > 0\} \tag{4.20}$$

denote the set of present clusters. For each $k \in \mathcal{C}_{\mathcal{B}}$, we define

$$\tilde{q}_k = \frac{q_k}{\sum_{j \in \mathcal{C}_{\mathcal{B}}} q_j}. \tag{4.21}$$

The cluster-DRO loss is then

$$\mathcal{L}_{\text{ClusterDRO}} = \sum_{k \in \mathcal{C}_{\mathcal{B}}} \tilde{q}_k \mathcal{L}_k. \tag{4.22}$$

The cluster weights are updated using the same exponentiated-gradient principle as in AttackDRO, but now at the cluster level rather than the attack level. Given a cluster-

level difficulty score

$$s_k = \frac{1}{|\mathcal{B}_k|} \sum_{i \in \mathcal{B}_k} s_i, \qquad k \in \mathcal{C}_{\mathcal{B}}, \tag{4.23}$$

the update increases the weight of clusters with higher average difficulty. This allows AttackDRO++ to emphasize hard latent groups even when they do not align exactly with the original attack identities.

Since model representations evolve during training, cluster assignments must also be updated. AttackDRO++ therefore maintains a feature bank $\mathcal{F}$, which stores detached augmented feature vectors from recent minibatches. When the refresh condition is met, K-means is refitted on $\mathcal{F}$, and subsequent minibatches are assigned using the updated clusterer. Before the first K-means refresh, cluster assignments are bootstrapped randomly.

The refresh condition is treated as a policy rather than a fixed part of the method. It can be defined in terms of epochs or training steps. In the main experiments, we use an epoch-based refresh schedule with $R = 2$ epochs. After each refresh, the feature bank is reset or updated according to the memory policy, and the cluster weights are refreshed according to the chosen q-refresh policy.

4.6 Augmented Clustering with Gradient Fingerprints

Representation-space clustering alone may not fully capture adversarial difficulty. Two adversarial examples can be close in feature space while inducing different optimization behavior. To capture this additional signal, AttackDRO++ constructs an augmented clustering feature that combines representation similarity, class information, and gradient-level training dynamics.

For an adversarial example x_i^{adv}, let

$$(z_i, h_i) = f_\theta(x_i^{\text{adv}})$$

denote the logits and penultimate-layer feature, and let

$$p_i = \text{softmax}(z_i)$$

be the predicted class distribution.

The first component is the normalized representation feature,

$$\bar{h}_i = \text{norm}(h_i),$$

which captures semantic similarity in the model feature space.

The second component is a class-aware label feature,

$$\lambda_{\text{lbl}}\text{onehot}(y_i),$$

which provides weak label structure during clustering.

The third component is the gradient fingerprint. For a linear classifier head

$$f_\theta(\cdot) = W\phi_\theta(\cdot) + b, \qquad W \in \mathbb{R}^{C\times d_h},$$

the per-example gradient of the cross-entropy loss with respect to the classifier weight matrix is

$$\nabla_W \ell(f_\theta(x_i^{\text{adv}}), y_i) = (p_i - \text{onehot}(y_i)) \otimes h_i \in \mathbb{R}^{C\times d_h}.$$

We use this quantity as a gradient proxy and denote it by

$$G_i = (p_i - \text{onehot}(y_i)) \otimes h_i.$$

Intuitively, G_i describes the training direction induced by sample i. Thus, two adversarial examples may be grouped together not only because they are close in representation space, but also because they induce similar optimization updates.

The raw gradient proxy has dimension Cd_h. For CIFAR-10 with a ResNet-18 backbone, this corresponds to $10 \times 512 = 5120$ dimensions. To make the fingerprint more compact and efficient for clustering, we use a fixed random projection matrix

$$P \in \mathbb{R}^{Cd_h \times d_{\text{proj}}},$$

and define the projected gradient fingerprint as

$$g_i = \text{vec}(G_i)P \in \mathbb{R}^{d_{\text{proj}}}.$$

In our default setting, $d_{\text{proj}} = 128$. The projection is fixed throughout training and introduces no additional learnable parameters.

The final augmented clustering feature is

$$\psi_i = \left[\text{norm}(h_i) \parallel \lambda_{\text{lbl}}\text{onehot}(y_i) \parallel \lambda_{\text{grad}}\text{norm}(g_i)\right].$$

The three components play complementary roles. The representation feature captures semantic similarity, the label feature provides weak class-aware structure, and the gradient fingerprint captures optimization-level similarity. This allows AttackDRO++ to discover latent adversarial groups that reflect both where examples lie in feature space and how they influence training.

4.7 Uniform-Anchored Training Objective

A pure Cluster-DRO objective can become unstable when the cluster weights $\mathbf{q}$ concentrate too strongly on a small number of hard clusters. We refer to this failure mode as *q-collapse*. In the extreme case, the objective approaches a worst-cluster loss, where most of the training signal is dominated by one cluster. This behavior is undesirable in multi-attack adversarial training because the hardest cluster may change after each cluster refresh, and over-emphasizing one cluster can reduce robustness on the remaining attacks or groups.

To stabilize training, AttackDRO++ uses a uniform-anchored Cluster-DRO objective that interpolates between the uniform multi-attack ERM loss and the adaptive cluster-DRO loss:

$$\mathcal{L}_{\text{AttackDRO++}} = (1 - \lambda_{\text{DRO}})\mathcal{L}_{\text{ERM}} + \lambda_{\text{DRO}}\mathcal{L}_{\text{ClusterDRO}}. \tag{4.24}$$

Here, $\lambda_{\text{DRO}} \in [0, 1]$ controls the strength of the Cluster-DRO correction. When $\lambda_{\text{DRO}} = 0$, the method reduces to uniform multi-attack ERM. When $\lambda_{\text{DRO}} = 1$, the method becomes pure Cluster-DRO.

Equivalently, Equation (4.24) can be written as

$$\mathcal{L}_{\text{AttackDRO++}} = \mathcal{L}_{\text{ERM}} + \lambda_{\text{DRO}}\left(\mathcal{L}_{\text{ClusterDRO}} - \mathcal{L}_{\text{ERM}}\right). \tag{4.25}$$

This form shows that λ_{DRO} scales the difference between the cluster-reweighted loss and the uniform loss. If the cluster-DRO loss is close to the uniform loss, the correction is small. If some clusters are substantially harder, the correction becomes larger, but its influence remains controlled by λ_{DRO}.

In the main experiments, we use $\lambda_{\text{DRO}} = 0.35$, referred to as the `Anchor35` setting. This value keeps the uniform multi-attack objective as the dominant stabilizing term while

still allowing the model to adapt toward harder clusters. Intuitively, this suggests an inverted-U behavior: a small amount of Cluster-DRO pressure can improve robustness by emphasizing hard groups, whereas excessive pressure may overfit unstable clusters and harm the balance across attacks.

A second stabilization mechanism is the floor projection applied to the cluster weights. After each exponentiated-gradient update, $\mathbf{q}$ is normalized and projected so that every cluster keeps at least $q_{\min}$ mass:

$$q_k \geq q_{\min}, \qquad \sum_{k=1}^{K} q_k = 1. \tag{4.26}$$

This prevents any cluster from being completely ignored. In our default setting, $q_{\min} = 0.03$. Together, the uniform anchor and the floor projection ensure that training receives both a stable per-sample ERM signal and an adaptive cluster-level correction.

Finally, the q-update is delayed during the early training phase. For the first T_w epochs, $\mathbf{q}$ remains uniform while the model representation and initial cluster assignments become more reliable. After this warmup period, the cluster weights are updated using the exponentiated-gradient rule. In the main configuration, $T_w = 3$ epochs.

4.8 Complete Training Framework

Algorithm 1 summarizes the complete AttackDRO++ training pipeline. The method combines mixed adversarial batch generation, augmented feature construction, cluster-level reweighting, q-updates, and a uniform-anchored loss. Table 4.1 lists the default hyperparameters used in the main experiments.

Table 4.1: Default hyperparameters for AttackDRO++ in the main experiments.

Hyperparameter	Symbol	Value	Role
DRO anchor strength	λ_{DRO}	0.35	Interpolates between uniform ERM and Cluster-DRO
Number of clusters	K	4	Controls latent group granularity
q learning rate	η_q	3×10^{-4}	Step size for exponentiated-gradient update
q-floor	$q_{\min}$	0.03	Minimum mass retained by each cluster
q-warmup	T_w	3 epochs	Delays adaptive reweighting until clusters are more reliable
Gradient projection dim.	d_{proj}	128	Dimension of projected gradient fingerprint
Gradient weight	λ_{grad}	0.1	Weight of GradFP component in ψ
Label weight	λ_{lbl}	0.1	Weight of label component in ψ
Feature bank size	$\lvert\mathcal{F}\rvert$	4,096	Buffer used to refit K-means
Cluster refresh interval	R	2 epochs	Default K-means refit frequency
q-refresh policy	–	reset	Resets $\mathbf{q}$ to uniform after cluster refresh
Bootstrap assignment	–	random	Used before the first K-means refit

Algorithm 1: AttackDRO++ with Gradient Fingerprints and Uniform Anchor

Input: Training set $\mathcal{D}_{\text{train}}$, model f_θ, source attacks $\mathcal{A}_{\text{src}} = \{A_1, \dots, A_M\}$, Cluster count K, anchor weight λ_{DRO}, learning rates η_θ, η_q, q-floor $q_{\min}$, Label weight λ_{lbl}, gradient weight λ_{grad}, projection dimension d_{proj}, Refresh interval R, warmup epochs T_w

Output: Trained model f_θ^*

Initialize $\mathbf{q} \leftarrow (1/K, \dots, 1/K)$
Initialize feature bank $\mathcal{F} \leftarrow \emptyset$
Initialize K-means clusterer as unavailable
Sample fixed projection matrix $P \in \mathbb{R}^{Cd_h \times d_{\text{proj}}}$
for *epoch* $e = 1, \dots, E$ **do**
    for *each minibatch* (x, y) *of size* N **do**
        Generate mixed adversarial batch x^{adv} using $\mathcal{A}_{\text{src}}$
        Compute logits and features $(z, h) \leftarrow f_\theta(x^{\text{adv}})$
        Build augmented features $\psi \leftarrow \text{BuildFeature}(z, h, y, P)$
        Assign cluster ids $c \leftarrow \text{AssignCluster}(\psi, \text{Clusterer}, K)$
        Compute uniform loss $\mathcal{L}_{\text{ERM}}$
        Compute cluster-DRO loss $\mathcal{L}_{\text{ClusterDRO}}$
        Compute final loss

$$\mathcal{L} = (1 - \lambda_{\text{DRO}})\mathcal{L}_{\text{ERM}} + \lambda_{\text{DRO}}\mathcal{L}_{\text{ClusterDRO}}.$$

        Update model parameters θ using SGD
        if $e > T_w$ **then**
            Update $\mathbf{q}$ using exponentiated gradient and floor projection
        end
        Add detached features ψ to feature bank $\mathcal{F}$
    end
    if *refresh condition is met* **then**
        Refresh K-means clusterer using $\text{KMeans}(\mathcal{F}, K)$
        Apply q-refresh policy to $\mathbf{q}$ `// default: reset to uniform`
        Reset or update feature bank $\mathcal{F}$ according to the memory policy
    end
end
return f_θ^*

The refresh condition is written as a policy rather than a fixed operation. It can be instantiated using either epoch-based or step-based schedules. Unless otherwise specified, the

main experiments use an epoch-based schedule with $R = 2$ epochs and reset $\mathbf{q}$ to the uniform distribution after each cluster refresh.

4.8.1 Progression from Stage 1 to the Final Method

This methodology directly extends the attacks-as-domains framework from Stage 1, which diagnosed the cross-attack robustness gap. Stage 2 moves from diagnosis to intervention. The progression is:

1. **Single-AT**: trains against one source attack and can leave a large cross-attack robustness gap.
2. **Uniform ERM**: trains on multiple source attacks with equal weighting; it is strong but cannot adapt to weak groups.
3. **AttackDRO**: reweights attack-level groups, but remains limited by the assumption that each attack group is homogeneous.
4. **AttackDRO++**: discovers latent cross-attack groups using K-means over adversarial representations.
5. **Gradient Fingerprints**: enrich clustering with optimization-level difficulty information.
6. **Uniform Anchor**: stabilizes Cluster-DRO by preserving the strong uniform multi-attack training signal.

The evaluation discipline from Stage 1 is preserved throughout: source attacks are fixed during training, heldout attacks are never used for optimization, and results are reported both per attack and through aggregate robustness metrics.

4.8.2 Implementation and Computational Considerations

The final implementation evaluated in the main experiments is AttackDRO++ with gradient fingerprints, an online K-means refresh policy, $K = 4$ clusters, and uniform-anchor strength $\lambda_{\text{DRO}} = 0.35$. In configuration files, this setting is recorded as `attackdro_pp_anchor035_gradfp_online_k4`. The identifier is used only as a reproducibility handle; the report-facing method name is AttackDRO++ (Ours).

The main additional state introduced by AttackDRO++ is the feature bank used for cluster refresh. The bank stores detached augmented features, so clustering does not back-propagate through past minibatches. K-means is refitted only at the configured refresh interval, and the cluster weights are reset or refreshed after the cluster identities change. This keeps the adaptive grouping mechanism tied to recent model representations while avoiding a per-step clustering cost.

The dominant computational cost remains adversarial example generation, because all compared multi-attack methods use the same mixed source attacks. The AttackDRO++ additions therefore change the training objective and bookkeeping rather than the threat model or evaluation protocol. Chapter 5 specifies the concrete training configuration, software stack, and evaluation procedures used to run this implementation. This chapter formulated cross-attack adversarial training as a domain-aware optimization problem and introduced the progression from Multi-AT to AttackDRO and AttackDRO++. The final method combines cluster-based grouping, gradient fingerprints, and a uniform anchor to emphasize difficult adversarial groups while retaining the stability of the uniform multi-attack baseline.

The chapter has defined the method and its training pipeline, but it has not yet established how the method behaves empirically. Chapter 5 therefore specifies the dataset, attacks, metrics, and evaluation protocols used to evaluate the proposed method.

Chapter 5

Experimental Setup

This chapter defines the reproducibility contract for the empirical analysis: dataset splits, model architecture, attack suite, training configuration, hyperparameters, statistical protocol, evaluation metrics, graybox transfer design, and experiment tracking. Results and interpretation are reserved for Chapter 6.

5.1 Dataset and Preprocessing

The experiments are conducted on CIFAR-10, a standard image-classification benchmark consisting of ten object categories. Each image has spatial resolution 32×32 with three color channels. The dataset is divided into training, validation, and test subsets. In our experimental pipeline, 40,000 images are used for training, 10,000 images are used for validation and checkpoint selection, and 10,000 images are reserved for final test-set evaluation.

The validation set is used only for model selection and hyperparameter monitoring. Final reported numbers are computed on the test set after the checkpoint has been selected. This separation is important because adversarial robustness estimates can vary across random seeds and training checkpoints; using a held-out validation set reduces the risk of selecting a checkpoint based on test-set performance. All images are normalized using the same preprocessing pipeline across methods. During training, standard CIFAR-10 data augmentation is applied to the clean images before adversarial examples are generated. The augmentation includes random cropping with padding and random horizontal flipping. During validation and testing, no stochastic augmentation is used. This ensures that differences between methods are caused by the training objective rather than by differences in preprocessing or evaluation.

The same dataset split, preprocessing pipeline, and evaluation dataloaders are used for all compared methods. This includes the single-attack PGD-AT and DDN-AT baselines, Multi-AT, AttackDRO, AttackDRO++, and AttackDRO++ (Ours).

5.2 Model Architecture

The primary architecture used in this study is ResNet-18, based on the residual learning framework of He et al. [1], adapted for CIFAR-10 resolution. ResNet-18 provides a useful balance between expressiveness and computational cost: it is large enough to support nontrivial adversarial training experiments, but still efficient enough to run repeated-seed comparisons and ablation studies.

The classifier is denoted by f_θ. For an input image x, the model outputs logits

$$z = f_\theta(x) \in \mathbb{R}^C,$$

where $C = 10$ is the number of CIFAR-10 classes. In addition to logits, the proposed method requires access to the penultimate-layer representation

$$h = \phi_\theta(x) \in \mathbb{R}^{d_h}.$$

This feature vector is used for clustering adversarial examples in AttackDRO++ and for constructing the gradient-fingerprint augmented feature described in Chapter 4. For ResNet-18 on CIFAR-10, the penultimate representation dimension is $d_h = 512$. Therefore, the raw classifier-head gradient proxy has dimension $Cd_h = 10 \times 512 = 5120$, which is later reduced by random projection.

Unless otherwise stated, all main experiments use the same ResNet-18 architecture and differ only in the training objective. This controlled setup ensures that performance differences can be attributed to the training method rather than to architectural changes. Higher-capacity architectures are treated as scaling checks or future extensions rather than the main basis for comparison.

5.3 Attack Suite and Configuration

Stage 2 evaluates whether robustness learned from a small set of source attacks generalizes to a broader attack suite. The attack suite is divided into two groups: source attacks used during training and evaluation attacks used only for validation or final testing.

5.3.1 Training Attacks

The main training setup uses two source attacks:

$$\mathcal{A}_{\text{src}} = \{\text{PGD-}\ell_\infty, \text{DDN-}\ell_2\}.$$

PGD-ℓ_∞ represents the standard first-order ℓ_∞ adversarial training setting. It uses cross-entropy loss and perturbation budget $\varepsilon_\infty = 8/255$. DDN-$\ell_2$ is based on the Decoupled Direction and Norm attack of Rony et al. [2]. DDN is an efficient gradient-based ℓ_2 attack that decouples the perturbation direction from its norm: the direction is updated using gradient information, while the perturbation norm is adapted depending on whether the current example is already adversarial.

Table 5.1 summarizes the source attacks used during training. Both attacks are used to generate mixed adversarial minibatches for Multi-AT, AttackDRO, AttackDRO++, and AttackDRO++ (Ours).

Table 5.1: Source attack configuration used during training. Both attacks are used to generate mixed adversarial minibatches for Multi-AT, AttackDRO, AttackDRO++, and AttackDRO++ (Ours).

Attack	Norm	Budget	Steps	Loss / objective	Role
PGD-ℓ_∞	ℓ_∞	$8/255$	20	Cross-entropy	Source attack
DDN-ℓ_2	ℓ_2	0.5	20	DDN objective	Source attack

For Multi-AT training, each minibatch is split across the source attacks and adversarial examples are generated using the assigned attack. The same mixed-batch source attack construction is also used by AttackDRO, AttackDRO++, and AttackDRO++ (Ours). Thus, the compared methods share the same adversarial data generation process and differ mainly in how they weight or group the generated examples.

5.3.2 Evaluation Attacks

The evaluation suite includes both source attacks and held-out attacks. The two source attacks used during training are also included in validation and test evaluation, while the remaining attacks are held out from training. This protocol directly tests the attacks-as-domains hypothesis: a robust model should not only perform well on the attacks used for optimization, but should also generalize to attacks with different perturbation geometries and optimization dynamics.

For validation and final testing, we evaluate each checkpoint using the attack suite in Table 5.2. The main aggregate metric, denoted Mean(8), is computed over the eight attacks in the table. AutoAttack is reported separately and is therefore not included in Mean(8).

Table 5.2: Validation and test attacks used to compute Mean(8), the average robust accuracy over the eight non-AutoAttack evaluation attacks. Source attacks are also used during training; held-out attacks are used only for validation and test evaluation.

Attack	Norm	Budget	Steps	Group
FGSM-RS	ℓ_∞	$8/255$	1	Held-out ℓ_∞
PGD-ℓ_∞	ℓ_∞	$8/255$	20	Source
TPGD	ℓ_∞	$8/255$	10	Held-out ℓ_∞
MI-FGSM	ℓ_∞	$8/255$	10	Held-out ℓ_∞
PGD-ℓ_2	ℓ_2	0.5	10	Held-out ℓ_2
DDN-ℓ_2	ℓ_2	0.5	20	Source
DeepFool-ℓ_2	ℓ_2	0.5	50	Held-out ℓ_2
CW-ℓ_2	ℓ_2	0.5	200	Held-out ℓ_2

For aggregate analysis, we report four summary metrics derived from the same eight attacks: Seen, Held-out ℓ_∞, Held-out ℓ_2, and Mean(8). Their exact definitions are given in Section 5.7.

AutoAttack configuration AutoAttack is used as an additional standardized ℓ_∞ stress test following Croce and Hein [3]. It is not included in Mean(8), because it is computationally more expensive and follows a separate evaluation protocol.

The AutoAttack configuration uses the standard ℓ_∞ setting with perturbation budget $\varepsilon_\infty = 8/255$. In early experiments and ablations, AutoAttack is sometimes evaluated on a subset of 512 test samples as a sanity check. These 512-sample results are useful for detecting severe robustness collapse, but they are not used as the primary ranking criterion because the sample size is smaller and the variance is higher. For the final stress test, AutoAttack is evaluated on the full test set for a fixed random-seed panel.

Table 5.3: AutoAttack-ℓ_∞ configuration used as a separate stress test. AutoAttack is reported separately from Mean(8).

Attack	Norm	Budget	Role
AutoAttack	ℓ_∞	$8/255$	Separate stress test

5.4 Training Configuration

All methods are trained under a controlled adversarial training pipeline. Unless otherwise stated, the optimizer, model architecture, dataset split, preprocessing pipeline, and evaluation protocol are kept fixed across methods. This ensures that performance differences mainly come from the training objective rather than from implementation or evaluation differences.

The primary training configuration is summarized in Table 5.4. All main experiments use ResNet-18 on CIFAR-10 with stochastic gradient descent (SGD), momentum, weight decay, mixed-precision training, and a multi-step learning-rate schedule. The main methods are trained for 50 epochs. During training, each minibatch contains a balanced mixture of source attacks. In the two-source setting, approximately half of the minibatch is assigned to PGD-ℓ_∞ and the other half to DDN-ℓ_2.

Table 5.4: General training configuration used for the main experiments.

Component	Setting	Description
Dataset	CIFAR-10	40k train / 10k validation / 10k test.
Backbone	ResNet-18	CIFAR-10 adapted residual network.
Training epochs	50	Main training budget.
Batch size	128	Used for all main methods.
Optimizer	SGD	Momentum-based stochastic gradient descent.
Momentum	0.9	Standard SGD momentum.
Weight decay	5×10^{-4}	Weight regularization.
Initial learning rate	0.1	Decayed during training.
Learning-rate schedule	Multi-step	Same schedule for all compared methods.
Mixed precision	Enabled	Reduces memory usage and training time.
Gradient clipping	Enabled	Stabilizes adversarial training.
Checkpoint selection	Validation robust average	Test set is used only after selection.

Checkpoint selection is based on validation robustness rather than test performance. Af-

ter training, the selected checkpoint is evaluated once on the test set. This protocol is applied consistently to the uniform baseline and all AttackDRO variants.

Table 5.5 summarizes the main training methods. All multi-attack methods use the same source attacks and mixed-batch adversarial example generation; they differ only in how adversarial examples are weighted or grouped during optimization.

Table 5.5: Summary of compared training methods.

Method	Grouping / weighting strategy	Experimental role
PGD-AT	One source attack only	Specialist ℓ_∞ baseline.
DDN-AT	One source attack only	Specialist ℓ_2 baseline.
Multi-AT	Uniform sample weighting over mixed PGD-ℓ_∞ and DDN-ℓ_2 batches	Main strong baseline.
AttackDRO	GroupDRO over attack identity	Coarse attack-level DRO baseline.
AttackDRO++	GroupDRO over discovered clusters	Cluster-based group discovery baseline.
AttackDRO++ (Ours)	Uniform-anchored Cluster-DRO with gradient fingerprints	Proposed method.

For the proposed method, cluster assignments are refreshed periodically using a feature bank. The default refresh interval is two epochs. The cluster weight vector $\mathbf{q}$ is updated only after a warmup period, so that early noisy representations do not dominate adaptive reweighting.

5.5 Hyperparameter Choices and Ablation Factors

Table 5.6: Default hyperparameters for AttackDRO++ (Ours).

Hyperparameter	Symbol	Default value	Purpose
Source attacks	$\mathcal{A}_{\text{src}}$	PGD-ℓ_∞, DDN-ℓ_2	Training attack domains.
Number of clusters	K	4	Latent group granularity.
DRO anchor strength	λ_{DRO}	0.35	Balances ERM and Cluster-DRO.
q learning rate	η_q	3×10^{-4}	Step size for q update.
q floor	$q_{\min}$	0.03	Prevents ignored clusters.
q warmup	T_w	3 epochs	Delays early reweighting.
Cluster refresh interval	R	2 epochs	K-means refresh frequency.
Feature bank size	$\lvert\mathcal{F}\rvert$	4096	Stores detached features.
Gradient projection dim.	d_{proj}	128	Compresses GradFP features.
Gradient feature weight	λ_{grad}	0.1	Weights GradFP features.
Label feature weight	λ_{lbl}	0.1	Weights label features.
Clusterer	–	K-means	Assigns latent groups.
q refresh policy	–	Reset after refresh	Avoids stale cluster weights.

The number of clusters is set to $K = 4$ as a moderate grouping resolution. Too few clusters may fail to capture meaningful difficulty structure, while too many clusters may fragment the minibatch and produce noisy group-loss estimates. The default anchor strength $\lambda_{\text{DRO}} = 0.35$ is chosen to keep the Multi-AT loss as the dominant stabilizing signal while still allowing cluster-level adaptive correction.

The q-floor and warmup settings are included for stability. The floor projection prevents any cluster from being completely ignored, while the warmup period prevents the exponentiated-gradient update from acting on unreliable early cluster assignments. The feature bank and refresh interval control how quickly the clusterer adapts to evolving model representations. In the default setting, refreshing every two epochs provides a balance between stability and adaptivity.

The gradient fingerprint is projected to $d_{\text{proj}} = 128$ dimensions for computational efficiency. For CIFAR-10 with ResNet-18, the classifier-head gradient proxy has raw dimension $Cd_h = 10 \times 512 = 5120$. The fixed random projection reduces this dimension while retaining useful information about the optimization direction induced by each adversarial example.

Table 5.7 lists the main hyperparameters varied in ablation experiments. These ablations are used in Chapter 6 to justify the default configuration and to separate the effect of the

proposed components from the strong uniform multi-attack baseline.

Table 5.7: Hyperparameters varied in ablation studies.

Ablation factor	Values / variants	Purpose
Anchor strength	$\lambda_{\text{DRO}} \in \{0.20, 0.35, 0.50\}$	Tests the uniform-anchor trade-off.
Cluster count	$K \in \{2, 4, 6, 10\}$	Tests sensitivity to group granularity.
q update	update q vs. uniform q	Tests adaptive reweighting.
Refresh schedule	1 epoch vs. 2 epochs	Tests cluster-refresh frequency.
Gradient fingerprint	With vs. without GradFP	Tests optimization-level features.
Random clusters	Learned vs. random assignment	Negative control for clustering.

5.6 Statistical Significance Protocol

To ensure that the reported improvements are not caused by seed-specific randomness, this study applies a repeated-seed statistical protocol. Each method is trained and evaluated under the same set of seeds, dataset split, architecture, attack suite, and evaluation configuration. The seed-level result is used as the basic unit of analysis.

For a given metric, let x_i be the performance of AttackDRO++ under seed i, and let y_i be the performance of a baseline method under the same seed. The paired difference is defined as:

$$d_i = x_i - y_i. \tag{5.1}$$

All statistical comparisons are performed on these paired differences. This paired design reduces the influence of random initialization, data shuffling, and optimization noise, allowing the analysis to focus on the relative performance difference between methods.

The primary significance test is the Wilcoxon signed-rank test. This test is selected because it is non-parametric and does not assume that the paired differences are normally distributed. This property is important for repeated-seed adversarial robustness experiments, where the number of seeds is limited and the distribution of performance differences may be irregular. For directional comparisons, the alternative hypothesis is that AttackDRO++ achieves higher performance than the baseline:

$$H_0 : \text{median}(d_i) \leq 0, \tag{5.2}$$

$$H_1 : \text{median}(d_i) > 0. \tag{5.3}$$

A paired t-test is also reported as a secondary parametric check. While the paired t-test assumes approximate normality of the paired differences, it provides a useful sanity check. When both the Wilcoxon signed-rank test and the paired t-test support the same conclusion, the result is considered more stable. If the two tests disagree, the Wilcoxon test is prioritized because it is less dependent on distributional assumptions.

In addition to p-values, this study reports the paired effect size using Cohen's d_z:

$$d_z = \frac{\bar{d}}{s_d}, \tag{5.4}$$

where $\bar{d}$ is the mean paired difference and s_d is the sample standard deviation of the paired differences. Cohen's d_z is used to quantify the magnitude of the improvement. Conventional thresholds are used as rough guidance, with 0.2, 0.5, and 0.8 corresponding to small, medium, and large effects, respectively. However, in adversarial robustness evaluation, even moderate effect sizes may be practically meaningful if the improvement is consistent across strong attacks or improves worst-case robustness.

To estimate uncertainty, 95% bootstrap confidence intervals are computed from seed-level results. For each method and metric, the seed-level values are resampled with replacement 10,000 times, and the mean is recomputed for each resample. The 2.5th and 97.5th percentiles of the bootstrap distribution are used as the confidence interval. Bootstrap confidence intervals are also computed for the paired difference $x_i - y_i$, as this directly estimates the uncertainty of the improvement over each baseline.

Because multiple attacks, metrics, and baseline comparisons are evaluated, Holm–Bonferroni correction is applied within each metric family. Both raw and corrected p-values are reported, but corrected p-values are used for final significance interpretation. This correction reduces the risk of false positive findings while remaining less conservative than the standard Bonferroni correction.

The reporting format used in the results chapter is summarized in Table 5.8.

Table 5.8: Reporting format for the statistical significance protocol.

Quantity	Reported value	Purpose
Mean performance	Mean across seeds	Reports average robustness.
Seed variation	Standard deviation	Measures run-to-run variability.
Uncertainty	95% bootstrap confidence interval	Estimates uncertainty of the mean.
Primary test	Wilcoxon signed-rank test	Non-parametric paired comparison.
Secondary test	Paired t-test	Parametric sanity check.
Effect size	Cohen's d_z	Measures magnitude of improvement.
Correction	Holm–Bonferroni correction	Controls multiple comparisons.

A comparison is considered statistically supported when AttackDRO++ improves the mean metric, obtains a corrected p-value below the significance threshold, and shows a non-trivial paired effect size. This protocol is applied consistently to white-box robustness, gray-box transfer robustness, aggregate robustness metrics, and ablation studies.

5.7 Evaluation Metrics and Robustness Definitions

This section defines the metrics used to compare all methods. Let

$$\mathcal{D}_{\text{test}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$$

denote the test set, and let A be an adversarial attack. The adversarial example generated by A is denoted by

$$\mathbf{x}_i^A = A(f_\theta, \mathbf{x}_i, y_i).$$

The robust accuracy under attack A is

$$\text{Acc}(A) = \frac{1}{n} \sum_{i=1}^{n} \mathbb{I}\left[\arg\max_c f_\theta(\mathbf{x}_i^A)_c = y_i\right]. \tag{5.5}$$

Clean accuracy is computed without adversarial perturbation:

$$\text{Acc}_{\text{clean}} = \frac{1}{n} \sum_{i=1}^{n} \mathbb{I}\left[\arg\max_c f_\theta(\mathbf{x}_i)_c = y_i\right]. \tag{5.6}$$

The primary aggregate metric is Mean(8), defined as the average robust accuracy over the eight non-AutoAttack evaluation attacks:

$$\text{Mean}(8) = \frac{1}{|\mathcal{A}_8|} \sum_{A \in \mathcal{A}_8} \text{Acc}(A), \tag{5.7}$$

where

$$\mathcal{A}_8 = \{\text{FGSM-RS}, \text{PGD-}\ell_\infty, \text{TPGD}, \text{MI-FGSM}, \text{PGD-}\ell_2, \text{DDN-}\ell_2, \text{DeepFool-}\ell_2, \text{CW-}\ell_2\}.$$

We also report the worst robust accuracy across the same eight attacks:

$$\text{Worst}(8) = \min_{A \in \mathcal{A}_8} \text{Acc}(A). \tag{5.8}$$

For finer analysis, we report seen-source, held-out ℓ_∞, and held-out ℓ_2 averages:

$$\text{Seen} = \frac{1}{2}\left[\text{Acc}(\text{PGD-}\ell_\infty) + \text{Acc}(\text{DDN-}\ell_2)\right], \tag{5.9}$$

$$\text{Heldout}_{\ell_\infty} = \frac{1}{3}\left[\text{Acc}(\text{FGSM-RS}) + \text{Acc}(\text{TPGD}) + \text{Acc}(\text{MI-FGSM})\right], \tag{5.10}$$

$$\text{Heldout}_{\ell_2} = \frac{1}{3}\left[\text{Acc}(\text{PGD-}\ell_2) + \text{Acc}(\text{DeepFool-}\ell_2) + \text{Acc}(\text{CW-}\ell_2)\right]. \tag{5.11}$$

AutoAttack is reported separately from Mean(8). Early ablations may use AutoAttack on 512 test samples as a sanity check, while final AutoAttack comparisons are reported on the full test set for a fixed random-seed panel.

For graybox evaluation, adversarial examples are generated using a surrogate model f_{θ_s} and evaluated on a target model f_{θ_t}:

$$\text{Acc}_{s\to t}(A) = \frac{1}{n}\sum_{i=1}^{n} \mathbb{I}\left[\arg\max_c f_{\theta_t}\left(A(f_{\theta_s}, \mathbf{x}_i, y_i)\right)_c = y_i\right]. \tag{5.12}$$

Thus, the same table supports whitebox, same-method transfer, and cross-method graybox evaluation.

Finally, for fine-grained analysis, each attack-class pair is treated as an evaluation group. For attack A and class c, the attack-class group accuracy is

$$\text{Acc}(A, c) = \frac{1}{|\mathcal{D}_c|}\sum_{(\mathbf{x}_i, y_i)\in\mathcal{D}_c} \mathbb{I}\left[\arg\max_j f_\theta(\mathbf{x}_i^A)_j = y_i\right], \tag{5.13}$$

where $\mathcal{D}_c$ is the subset of test examples whose label is c.

Table 5.9: Summary of evaluation metrics.

Metric	Definition	Purpose
Clean accuracy	Accuracy on unperturbed test images	Measures standard classification performance.
Per-attack robust accuracy	Accuracy under a single adversarial attack	Shows which attacks each method resists or fails against.
Mean(8)	Average accuracy over the eight non-AutoAttack attacks	Main aggregate robustness metric.
Worst(8)	Minimum accuracy over the eight non-AutoAttack attacks	Measures the weakest attack in the suite.
Seen-source mean	Average over PGD-ℓ_∞ and DDN-ℓ_2	Measures robustness on source attacks.
Held-out ℓ_∞ mean	Average over FGSM-RS, TPGD, and MI-FGSM	Measures transfer to unseen ℓ_∞ attacks.
Held-out ℓ_2 mean	Average over PGD-ℓ_2, DeepFool-ℓ_2, and CW-ℓ_2	Measures transfer to unseen ℓ_2 attacks.
AutoAttack	Standard ℓ_∞ AutoAttack robust accuracy	Separate robustness stress test.
Graybox transfer accuracy	Target accuracy on adversarial examples generated by a surrogate	Measures transfer robustness across models.
Attack-class group accuracy	Accuracy for each attack $\times$ class pair	Diagnoses fine-grained failure modes.

5.8 Graybox Transfer Protocol

To evaluate whether the proposed method changes the structure of learned robustness rather than only improving whitebox accuracy, we conduct a cross-model graybox transfer experiment. The experiment uses four method families: PGD-AT, DDN-AT, Multi-AT, and AttackDRO++ (Ours). Each method is evaluated over five random seeds, giving 20 random-seed checkpoints in total.

Each checkpoint is used in two roles. As a *surrogate*, it generates adversarial examples. As a *target*, it is evaluated on adversarial examples generated by all surrogate checkpoints. Therefore, the full protocol evaluates

$$20 \times 20 = 400$$

surrogate–target pairs. This design produces whitebox, same-method transfer, and cross-method graybox transfer results from a single unified evaluation table.

To keep the computation tractable while still supporting per-class and per-attack analysis, each surrogate checkpoint generates a balanced adversarial cache. The CIFAR-10 test set contains 1000 images per class. For each class, the test images are split evenly across the eight evaluation attacks. Thus, each attack-class cell contains 125 source images:

$$10 \text{ classes} \times 8 \text{ attacks} \times 125 \text{ examples} = 10{,}000$$

adversarial examples per surrogate checkpoint.

The class-wise split is generated deterministically using a fixed shuffle seed. Therefore, the same attack-class cell always refers to the same underlying CIFAR-10 source images across surrogate checkpoints. This makes paired comparisons between surrogate and target methods well aligned.

For each surrogate–target pair, the evaluation produces one row per attack-class group, indexed by surrogate method, surrogate seed, target method, target seed, attack, and class. Since there are 80 attack-class groups per pair, the full table contains $400 \times 80 = 32{,}000$ rows. Transfer matrices, attack-class drilldowns, and paired method comparisons are all computed by aggregating this same long-form table. Table 5.10 summarizes the transfer regimes used in the analysis.

Table 5.10: Transfer regimes in the graybox evaluation protocol.

Regime	Condition	Rows	Purpose
Whitebox	Same method and same seed	$20 \times 80 = 1{,}600$	Recovers standard whitebox robustness as a sanity check.
Same-method transfer	Same method, different seed	$80 \times 80 = 6{,}400$	Measures seed-to-seed transfer within a method family.
Cross-method same-seed	Different methods, same seed	$60 \times 80 = 4{,}800$	Measures transfer between method families under matched seeds.
Cross-method cross-seed	Different methods, different seeds	$240 \times 80 = 19{,}200$	Measures the most general cross-method graybox transfer setting.

In Chapter 6, the cross-method same-seed and cross-method cross-seed regimes are combined to form the main graybox transfer analysis. The whitebox and same-method transfer regimes are reported separately as diagnostic checks.

5.9 Tools, Platforms, and Experiment Tracking

The experiments in this report were implemented in Python using a PyTorch-based training pipeline. The codebase is organized around a configurable experiment script that defines dataset construction, model initialization, adversarial attack generation, training objectives, checkpoint selection, and fixed evaluation scripts in one place. This organization is important because the compared methods differ mainly in their grouping and weighting strategies, while the dataset split, architecture, source attacks, evaluation attacks, optimizer, and checkpointing rules are kept fixed.

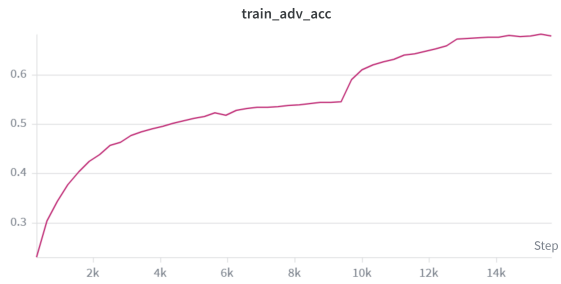
Table 5.11 summarizes the main tools and platforms used in the experimental workflow. Google Colab and Google Drive are used for interactive execution and persistent storage of checkpoints, cached adversarial examples, figures, and result files. PyTorch and torchvision provide the core deep-learning infrastructure, including model definitions, data loading, and optimization. The attack implementations are built primarily with TorchAttacks, with AutoAttack-ℓ_∞ and DDN-ℓ_2 handled through their dedicated packages when required. scikit-learn is used for K-means clustering in AttackDRO++, while NumPy and pandas are used to aggregate seed-level and attack-level results.

Table 5.11: Tools and platforms used in the experimental workflow. The table lists only tools that are part of the implemented training, evaluation, logging, or reporting pipeline.

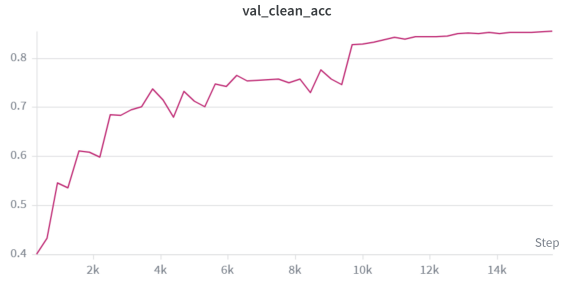
Tool / platform	Role	Use in this report
Google Colab	Interactive execution environment	Used to run training and evaluation notebooks with GPU support.
Google Drive	Persistent experiment storage	Stores checkpoints, cached adversarial examples, result tables, figures, and intermediate analysis files.
Python	Main implementation language	Used for the full training, evaluation, analysis, and plotting pipeline.
PyTorch	Deep-learning framework	Implements model training, automatic differentiation, mixed-precision execution, and checkpoint loading.
torchvision	Dataset and model utilities	Provides CIFAR-10 loading, image transforms, and ResNet backbones adapted for small images.
TorchAttacks	Adversarial attack library	Provides most gradient-based and optimization-based attacks used in training and evaluation.
AutoAttack package	Standardized robustness stress test	Used for AutoAttack-ℓ_∞ evaluation with perturbation budget $8/255$.
adv-lib	DDN attack implementation	Used for DDN-ℓ_2 adversarial example generation.
scikit-learn	Clustering utilities	Provides MiniBatch K-means for cluster discovery in AttackDRO++.
NumPy and pandas	Result processing	Used to aggregate per-seed, per-attack, per-class, and graybox transfer results.
matplotlib and SciencePlots	Visualization	Used to generate plots and publication-style figures for analysis.
Weights & Biases	Experiment tracking	Logs configurations, training curves, validation metrics, group weights, and diagnostic figures when enabled.

Experiment tracking is treated as part of the reproducibility protocol rather than as a separate result source. Each run records the method name, seed, dataset, backbone, source attacks, evaluation attacks, and key hyperparameters. During training, the pipeline logs loss, adversarial accuracy, validation robustness, learning rate, checkpoint state, and group-weight diagnostics when applicable. Figure 5.1 provides representative W&B exports for the core tracking metrics and the adaptive group-weight trajectory used as an AttackDRO++ diagnostic.

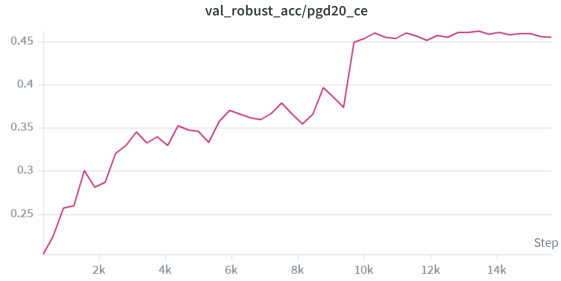
(a) Training loss.



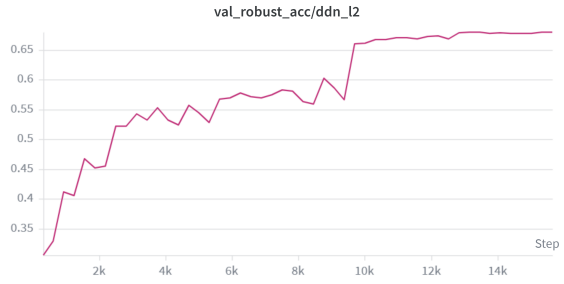
(b) Training accuracy.



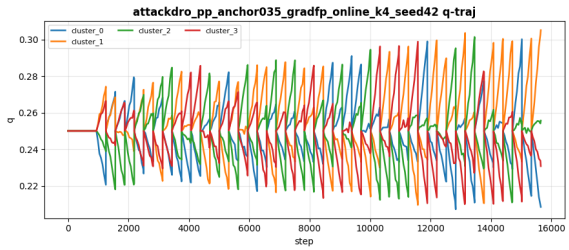
(c) Clean validation accuracy.



(d) PGD-ℓ_∞ validation accuracy.



(e) DDN-ℓ_2 validation accuracy.



(f) Group-weight trajectory.

Figure 5.1: Representative W&B dashboard exports for experiment tracking. Panels (a)–(e) track training loss, training accuracy, clean validation accuracy, and validation robustness under PGD-ℓ_∞ and DDN-ℓ_2 during a training run. Panel (f) records the AttackDRO++ group-weight trajectory diagnostic.

All test-set claims in Chapter 6 are based on saved checkpoints that are loaded after training and evaluated using fixed evaluation scripts. The main result tables are produced from structured result files rather than from manually copied console logs. This separation reduces the risk of mixing validation, test, whitebox, graybox, and diagnostic metrics. It also makes it possible to reuse the same result files for aggregate metrics, per-attack analysis, per-class analysis, and paired statistical comparisons.

Summary. Table 5.12 summarizes the full experimental setup used as the basis for the result claims in Chapter 6.

Table 5.12: Summary of the experimental setup used in this report. Unless otherwise stated, evaluations use CIFAR-10, ResNet-18, and the eight non-AutoAttack attacks for Mean(8), with AutoAttack-ℓ_∞ reported separately.

Component	Configuration
Dataset	CIFAR-10 with 40,000 training images, 10,000 validation images, and 10,000 test images.
Backbone	ResNet-18 adapted for CIFAR-10 resolution.
Main compared methods	PGD-AT, DDN-AT, Multi-AT, AttackDRO, AttackDRO++, and AttackDRO++ (Ours).
Source attacks	PGD-ℓ_∞ and DDN-ℓ_2.
Evaluation attacks for Mean(8)	FGSM-RS, PGD-ℓ_∞, TPGD, MI-FGSM, PGD-ℓ_2, DDN-ℓ_2, DeepFool-ℓ_2, and CW-ℓ_2.
Separate stress test	AutoAttack-ℓ_∞, reported separately from Mean(8).
Primary aggregate metric	Mean robust accuracy over the eight non-AutoAttack evaluation attacks.
Secondary metrics	Worst robust accuracy over the same eight attacks, seen-source robustness, held-out ℓ_∞ robustness, held-out ℓ_2 robustness, per-attack accuracy, per-class accuracy, and graybox transfer accuracy.
Repeated-seed protocol	Aggregate whitebox tables use paired random-seed comparisons; class-wise and graybox panels use five random seeds per method.
Graybox protocol	Four method families are evaluated across five random seeds each, giving 20 random-seed checkpoints and 400 surrogate–target pairs.
Statistical analysis	Paired random-seed comparisons with Wilcoxon signed-rank tests, paired t-tests, bootstrap confidence intervals, effect sizes, and Holm–Bonferroni correction.
Implementation stack	PyTorch, torchvision, TorchAttacks, AutoAttack package, adv-lib, scikit-learn, NumPy, pandas, matplotlib, SciencePlots, W&B, Google Colab, and Google Drive.

This chapter has defined the experimental setting used throughout the remainder of the report: the dataset, model, attacks, training configurations, hyperparameters, statistical protocol, evaluation metrics, graybox transfer design, and implementation tools. These definitions provide the reproducibility contract for the empirical claims that follow. Chapter 6 now presents the results obtained under this protocol and analyzes how the proposed method behaves under whitebox, graybox, per-class, and ablation settings.

Chapter 6

Results and Analysis

6.1 Whitebox Robustness Under Direct Attacks

6.1.1 Aggregate Comparison Across Methods

Table 6.1: Aggregate robust accuracy (%) on the CIFAR-10 evaluation suite, reported as mean $\pm$ standard deviation across 20 random-seed runs. Mean robust accuracy denotes Mean(8), the average robust accuracy over the eight non-AutoAttack evaluation attacks. Worst-case robust accuracy denotes Worst(8), the lowest robust accuracy among those same eight attacks. The held-out attack mean averages FGSM-RS, TPGD, MI-FGSM, PGD-ℓ_2, DeepFool-ℓ_2, and CW-ℓ_2. The held-out ℓ_∞ mean averages FGSM-RS, TPGD, and MI-FGSM; the held-out ℓ_2 mean averages PGD-ℓ_2, DeepFool-ℓ_2, and CW-ℓ_2. The seen-source mean averages PGD-ℓ_∞ and DDN-ℓ_2. AutoAttack is reported separately and is not included in Mean(8). Bold values indicate the best method for each metric.

Metric	PGD-AT	DDN-AT	Multi-AT	AttackDRO++ (Ours)
Mean robust accuracy	60.499 ± 0.327	56.226 ± 0.233	62.045 ± 0.447	**62.324 ± 0.147**
Held-out attack mean	62.268 ± 0.390	59.628 ± 0.254	64.083 ± 0.505	**64.401 ± 0.155**
Held-out ℓ_∞ mean	**62.515 ± 0.334**	51.594 ± 0.306	60.916 ± 0.427	61.149 ± 0.170
Held-out ℓ_2 mean	62.021 ± 0.474	**67.661 ± 0.278**	67.249 ± 0.603	67.654 ± 0.223
Seen-source mean	55.192 ± 0.182	46.020 ± 0.247	55.934 ± 0.303	**56.093 ± 0.166**
Worst-case robust accuracy	**49.656 ± 0.161**	25.957 ± 0.378	45.082 ± 0.281	45.079 ± 0.237

We compare four methods on 20 paired random-seed runs: PGD-AT, DDN-AT, Multi-AT, and AttackDRO++ (Ours). AttackDRO++ improves Mean(8) over Multi-AT by $+0.279$ pp ($p = 0.0076$, 14/20 wins), with the strongest channel on the held-out ℓ_2 mean ($+0.405$ pp, $p = 0.0079$, 16/20). It improves Mean(8) over PGD-AT by $+1.826$ pp

and over DDN-AT by $+6.098$ pp, both at $p < 10^{-10}$ on all 20 seeds, and achieves 2.5–3.3$\times$ tighter seed-to-seed standard deviation than Multi-AT on every average-case metric. Worst(8) and AutoAttack are statistically indistinguishable from Multi-AT ($|\Delta| < 0.005$ pp, $p > 0.97$): AttackDRO++ improves average and held-out robustness without disturbing standardized worst-case behavior. The single-attack baselines remain stronger than AttackDRO++ on the family they train against (PGD-AT on the held-out ℓ_∞ mean, DDN-AT on the held-out ℓ_2 mean) but lose substantially on the other, confirming the specialist-versus-balanced interpretation that motivates multi-attack training.

AttackDRO++ achieves the highest Mean(8), held-out attack mean, and seen-source mean. PGD-AT wins the held-out ℓ_∞ mean and Worst(8); DDN-AT wins the held-out ℓ_2 mean. The bold pattern visualizes the specialist-versus-balanced interpretation: each single-AT method dominates one column, while AttackDRO++ is best on every column-aggregating metric.

Effect-size summary. Table 6.2 reports paired effect magnitudes between AttackDRO++ and each baseline. Cohen's $d_z = \Delta/\sigma_\Delta$ is the standardized paired effect size; conventional thresholds are 0.2 (small), 0.5 (medium), 0.8 (large). Statistical significance is reported in Table 6.3.

Table 6.2: Effect-size summary for AttackDRO++ (Ours) against each baseline across 20 paired random-seed runs. Δ in percentage points; negative values mean the baseline is stronger.

	vs. Multi-AT		vs. PGD-AT		vs. DDN-AT	
Metric	Δ (pp)	d_z	Δ (pp)	d_z	Δ (pp)	d_z
Mean(8)	$+0.279$	0.67	$+1.826$	5.98	$+6.098$	23.09
Held-out attack mean	$+0.319$	0.67	$+2.134$	5.65	$+4.774$	16.91
Held-out ℓ_∞ mean	$+0.233$	0.57	-1.366	-4.27	$+9.555$	25.44
Held-out ℓ_2 mean	$+0.405$	0.66	$+5.633$	11.64	-0.008	-0.03
Seen-source mean	$+0.159$	0.55	$+0.901$	4.59	$+10.073$	31.72
Worst(8)	-0.003	-0.01	-4.576	-15.36	$+19.122$	36.91

Two regimes are visible. Against Multi-AT, every average-case metric is a medium-effect gain ($d_z \in [0.55, 0.67]$, $\Delta \in [+0.16, +0.41]$ pp) with Worst(8) at zero effect: AttackDRO++ shifts the average-case distribution by a small but consistent amount without disturbing worst-case behavior. Against the single-AT baselines, d_z exceeds 4 on most

metrics but flips negative on the family each specialist trains against. The medium-effect Multi-AT comparison validates the cluster-DRO objective as a refinement of Multi-AT; the large-but- asymmetric single-AT comparisons quantify the specialist tradeoff that motivates the multi-attack setting in the first place.

Paired aggregate robustness comparisons. Table 6.3 separates the main Multi-AT comparison from the single-attack specialist comparisons. AutoAttack is excluded from this table and reported separately in Section 6.1.3.

Table 6.3: Paired aggregate robustness comparison between AttackDRO++ (Ours) and each baseline across 20 paired random-seed runs. Δ is computed as AttackDRO++ minus the corresponding baseline in percentage points. The p-value is from a two-sided paired t-test.

Baseline	Metric	n	Δ (pp)	σ_Δ	Wins	d_z	p
AttackDRO++ (Ours) vs. Multi-AT							
Multi-AT	Mean(8)	20	$+0.279$	0.417	$14/20$	0.668	0.0076
Multi-AT	Held-out attack mean	20	$+0.319$	0.478	$13/20$	0.667	0.0076
Multi-AT	Held-out ℓ_∞ mean	20	$+0.233$	0.412	$14/20$	0.566	0.0204
Multi-AT	Held-out ℓ_2 mean	20	$+0.405$	0.609	$16/20$	0.664	0.0079
Multi-AT	Seen-source mean	20	$+0.159$	0.291	$15/20$	0.546	0.0246
Multi-AT	Worst(8)	20	-0.003	0.369	$9/20$	-0.008	0.971
AttackDRO++ (Ours) vs. PGD-AT							
PGD-AT	Mean(8)	20	$+1.826$	0.305	$20/20$	5.980	$< 10^{-10}$
PGD-AT	Held-out attack mean	20	$+2.134$	0.378	$20/20$	5.650	$< 10^{-10}$
PGD-AT	Held-out ℓ_∞ mean	20	-1.366	0.320	$0/20$	-4.273	$< 10^{-10}$
PGD-AT	Held-out ℓ_2 mean	20	$+5.633$	0.484	$20/20$	11.638	$< 10^{-20}$
PGD-AT	Seen-source mean	20	$+0.901$	0.197	$20/20$	4.585	$< 10^{-10}$
PGD-AT	Worst(8)	20	-4.576	0.298	$0/20$	-15.364	$< 10^{-20}$
AttackDRO++ (Ours) vs. DDN-AT							
DDN-AT	Mean(8)	20	$+6.098$	0.264	$20/20$	23.087	$< 10^{-20}$
DDN-AT	Held-out attack mean	20	$+4.774$	0.282	$20/20$	16.910	$< 10^{-20}$
DDN-AT	Held-out ℓ_∞ mean	20	$+9.555$	0.376	$20/20$	25.440	$< 10^{-20}$
DDN-AT	Held-out ℓ_2 mean	20	-0.008	0.319	$10/20$	-0.025	0.914
DDN-AT	Seen-source mean	20	$+10.073$	0.318	$20/20$	31.723	$< 10^{-20}$
DDN-AT	Worst(8)	20	$+19.122$	0.518	$20/20$	36.909	$< 10^{-20}$

Against Multi-AT, AttackDRO++ gives small but statistically supported gains on all

average-case metrics, while Worst(8) is unchanged. Against PGD-AT, AttackDRO++ improves Mean(8) and held-out ℓ_2 robustness, but PGD-AT remains stronger on held-out ℓ_∞ and Worst(8). Against DDN-AT, AttackDRO++ improves all ℓ_∞-related aggregates while matching DDN-AT on held-out ℓ_2. This again supports the main interpretation that single-attack baselines are specialists, while AttackDRO++ provides better average cross-attack balance.

Table 6.4: Multiple-comparison-corrected paired significance tests for AttackDRO++ (Ours) against each baseline across $n = 20$ paired seeds. Δ is computed as AttackDRO++ minus the corresponding baseline in percentage points. The table reports bootstrap 95% confidence intervals, seed-level win counts, Cohen's paired effect size d_z, paired t-test p-values, Wilcoxon signed-rank p-values, and Holm-corrected Wilcoxon p-values. Holm correction is applied within each baseline comparison across the seven reported metrics. AutoAttack is excluded from this table and reported separately in Section 6.1.3.

Baseline	Metric	Δ (pp)	95% CI (pp)	Wins	d_z	p_t	p_W	$p_{W,\text{Holm}}$
AttackDRO++ (Ours) vs. Multi-AT								
Multi-AT	Clean	+0.539	[+0.195, +0.903]	13/20	0.642	0.0098	0.0441	0.1449
Multi-AT	Mean(8)	+0.279	[+0.108, +0.463]	14/20	0.668	0.0076	0.0136	0.0953
Multi-AT	Held-out attack mean	+0.319	[+0.123, +0.527]	13/20	0.667	0.0076	0.0192	0.0962
Multi-AT	Held-out ℓ_∞ mean	+0.233	[+0.060, +0.410]	14/20	0.566	0.0204	0.0362	0.1449
Multi-AT	Held-out ℓ_2 mean	+0.405	[+0.144, +0.661]	16/20	0.664	0.0079	0.0136	0.0953
Multi-AT	Seen-source mean	+0.159	[+0.038, +0.285]	15/20	0.546	0.0246	0.0419	0.1449
Multi-AT	Worst(8)	-0.003	[-0.168, +0.155]	9/20	-0.008	0.9714	0.9839	0.9839
AttackDRO++ (Ours) vs. PGD-AT								
PGD-AT	Clean	+4.015	[+3.666, +4.386]	20/20	4.779	< 0.0001	< 0.0001	< 0.0001
PGD-AT	Mean(8)	+1.826	[+1.702, +1.965]	20/20	5.980	< 0.0001	< 0.0001	< 0.0001
PGD-AT	Held-out attack mean	+2.134	[+1.980, +2.303]	20/20	5.650	< 0.0001	< 0.0001	< 0.0001
PGD-AT	Held-out ℓ_∞ mean	-1.366	[-1.496, -1.223]	0/20	-4.273	< 0.0001	< 0.0001	< 0.0001
PGD-AT	Held-out ℓ_2 mean	+5.633	[+5.441, +5.849]	20/20	11.638	< 0.0001	< 0.0001	< 0.0001
PGD-AT	Seen-source mean	+0.901	[+0.815, +0.984]	20/20	4.585	< 0.0001	< 0.0001	0.0002
PGD-AT	Worst(8)	-4.576	[-4.708, -4.454]	0/20	-15.364	< 0.0001	< 0.0001	0.0002
AttackDRO++ (Ours) vs. DDN-AT								
DDN-AT	Clean	-4.563	[-4.712, -4.414]	0/20	-13.134	< 0.0001	< 0.0001	< 0.0001
DDN-AT	Mean(8)	+6.098	[+5.986, +6.209]	20/20	23.087	< 0.0001	< 0.0001	< 0.0001
DDN-AT	Held-out attack mean	+4.774	[+4.654, +4.893]	20/20	16.910	< 0.0001	< 0.0001	< 0.0001
DDN-AT	Held-out ℓ_∞ mean	+9.555	[+9.399, +9.718]	20/20	25.440	< 0.0001	< 0.0001	< 0.0001
DDN-AT	Held-out ℓ_2 mean	-0.008	[-0.143, +0.127]	10/20	-0.025	0.9137	0.9563	0.9563
DDN-AT	Seen-source mean	+10.072	[+9.938, +10.208]	20/20	31.723	< 0.0001	< 0.0001	0.0002
DDN-AT	Worst(8)	+19.122	[+18.904, +19.344]	20/20	36.909	< 0.0001	< 0.0001	< 0.0001

Table 6.3 reports the paired t-test comparison. To align with the statistical protocol described in Chapter 5, Table 6.4 additionally reports Wilcoxon signed-rank tests with Holm correction, bootstrap confidence intervals, paired effect sizes, and seed-level win counts. The multiple-comparison-corrected tests provide a conservative check on the paired t-test results. Against Multi-AT, the key question is whether the small positive gains on average-case metrics remain supported after Wilcoxon testing and Holm correction. Clean accuracy is included as a sanity metric, but the main robustness claims are based on Mean(8), held-out metrics, seen-source mean, and Worst(8). Against the single-attack baselines, the same analysis mainly verifies the specialist-versus-balanced pattern: PGD-AT remains stronger on ℓ_∞-dominated metrics, DDN-AT remains competitive on held-out ℓ_2, and AttackDRO++ provides stronger average cross-attack balance.

After Holm correction, the Multi-AT comparison should be interpreted cautiously. AttackDRO++ shows small positive differences over Multi-AT on Clean, Mean(8), held-out attack mean, held-out ℓ_∞ mean, held-out ℓ_2 mean, and seen-source mean, and these differences are supported by the paired t-tests, raw Wilcoxon tests, confidence intervals, and moderate paired effect sizes. However, the Holm-corrected Wilcoxon p-values for these metrics do not remain below the 0.05 threshold. Therefore, relative to Multi-AT, the evidence is best described as a small positive trend rather than a uniformly correction-robust improvement. Worst(8) is essentially unchanged, indicating that AttackDRO++ preserves the worst-case attack performance of Multi-AT rather than improving it.

For the single-attack baselines, the corrected tests support the expected specialist-versus-balanced interpretation. Compared with PGD-AT, AttackDRO++ achieves correction-robust gains on Mean(8), held-out attack mean, held-out ℓ_2 mean, and seen-source mean, but remains weaker on the held-out ℓ_∞ mean and Worst(8), consistent with PGD-AT's specialization toward ℓ_∞ robustness. Compared with DDN-AT, AttackDRO++ obtains correction-robust gains on Mean(8), held-out attack mean, held-out ℓ_∞ mean, seen-source mean, and Worst(8), while the held-out ℓ_2 mean is statistically indistinguishable. Overall, Table 6.4 suggests that AttackDRO++ mainly improves average cross-attack balance and held-out robustness, while its worst-case behavior depends on the baseline being compared.

6.1.2 Per-Attack Accuracy: Where Do Methods Diverge?

The aggregate metrics show the overall trend, but they do not reveal which attacks drive the improvement. Table 6.5 reports per-attack paired comparisons between AttackDRO++ and each baseline. The table is divided into three panels: the main comparison against Multi-AT, followed by the two single-attack specialist comparisons.

Table 6.5: Per-attack robust accuracy (%) for all four methods across 20 paired random-seed runs. Bold cells indicate the highest accuracy for each attack. The right-hand Δ and p columns report the paired comparison of AttackDRO++ (Ours) against Multi-AT. AttackDRO++ (Ours) wins the highest score on 4 of 8 attacks; the two single-attack baselines together account for the remaining 4, each winning on the family they were trained against. Δ values use the bold-marker convention * for $p < 0.05$ and ** for $p < 0.01$.

Attack	Family	PGD-AT	DDN-AT	Multi-AT	AttackDRO++ (Ours)	Δ vs. Multi-AT	p
FGSM-RS	held-out ℓ_∞	67.896	64.834	68.483	**68.939**	$+0.456^{**}$	0.0034
PGD-ℓ_∞	seen ℓ_∞	**49.656**	25.957	45.082	45.079	-0.003	0.971
TPGD	held-out ℓ_∞	**68.388**	59.310	66.934	67.043	$+0.109$	0.513
MI-FGSM	held-out ℓ_∞	**51.260**	30.638	47.331	47.466	$+0.135$	0.116
PGD-ℓ_2	held-out ℓ_2	64.319	**69.581**	69.197	69.571	$+0.374^{*}$	0.0120
DDN-ℓ_2	seen ℓ_2	60.728	66.083	66.785	**67.107**	$+0.321^{*}$	0.0136
DeepFool-ℓ_2	held-out ℓ_2	62.533	**68.290**	67.301	67.805	$+0.504^{**}$	0.0082
CW-ℓ_2	held-out ℓ_2	59.211	65.113	65.249	**65.585**	$+0.336^{**}$	0.0097

Table 6.5 disaggregates the Mean(8) result into the eight evaluation attacks. AttackDRO++ achieves the highest per-attack accuracy on four of eight attacks, including all three held-out ℓ_2 attacks where it beats Multi-AT at $p < 0.05$. PGD-AT wins on the three held-out ℓ_∞ attacks and on the seen-source attack PGD-ℓ_∞ that it trains directly against; DDN-AT wins on PGD-ℓ_2 and DeepFool-ℓ_2, the held-out ℓ_2 attacks closest to its training distribution. The pattern is consistent with a specialist interpretation of single-attack training: each single-AT method dominates a narrow attack family at the cost of substantial regression on the other. AttackDRO++'s improvement over Multi-AT is largest on held-out ℓ_2 attacks ($+0.374$ to $+0.504$ pp, all $p < 0.05$) and smallest on the seen ℓ_∞ attack (-0.003 pp), which explains why Worst(8) is preserved rather than improved.

6.1.3 Robustness Under the Hardest Evaluation Cases

This subsection examines two complementary stress indicators: the lowest robust accuracy across the eight main evaluation attacks and the separate AutoAttack-ℓ_∞ evaluation.

Two metrics test standardized worst-case behavior. Worst(8) is the minimum robust accuracy across the eight evaluation attacks per checkpoint; AutoAttack-ℓ_∞ is an ensemble ℓ_∞ stress test following Croce and Hein [3]. Tables 6.6 and 6.7 report each metric separately because they have different evaluation panels: Worst(8) is computed across the full $n = 20$ seed panel, while AutoAttack is reported in two layers — a 512-sample sanity evaluation across all $n = 20$ seeds and a full-test evaluation on the selected $n = 5$

seed panel.

Table 6.6: Worst(8) and 512-sample AutoAttack sanity results across 20 paired random-seed runs. Worst(8) is the minimum robust accuracy over the eight non-AutoAttack evaluation attacks. For AttackDRO++ − Multi-AT, the paired differences are -0.003 pp on Worst(8) ($p = 0.971$) and -0.001 pp on AutoAttack ($p = 0.998$). Bold values indicate the highest method.

	Worst(8)		AutoAttack-ℓ_∞	
Method	Mean (%)	σ (pp)	Mean (%)	σ (pp)
PGD-AT	**49.656**	0.161	**44.912**	1.136
DDN-AT	25.957	0.378	23.809	0.796
Multi-AT	45.082	0.281	41.318	1.167
AttackDRO++ (Ours)	45.079	0.237	41.317	1.050

Table 6.7: Full-test AutoAttack-ℓ_∞ results on the selected $n = 5$ paired seed panel. The paired difference is AttackDRO++ − Multi-AT: -0.006 pp, with $p = 0.9584$.

Method	Mean (%)	σ (pp)
Multi-AT	42.47	0.30
AttackDRO++ (Ours)	42.46	0.20

AttackDRO++ vs. Multi-AT. AttackDRO++ is statistically indistinguishable from Multi-AT on both worst-case metrics. On the $n = 20$ sanity panel, the paired differences are -0.003 pp for Worst(8) ($p = 0.971$) and -0.001 pp for AutoAttack ($p = 0.998$). The full-test AutoAttack panel gives the same conclusion (-0.006 pp, $p = 0.9584$). Thus, the claim is preservation rather than improvement: AttackDRO++ improves average and held-out robustness without degrading standardized worst-case behavior.

PGD-AT specialization. PGD-AT obtains the highest Worst(8) and AutoAttack because both metrics are dominated by ℓ_∞ robustness, the threat family it trains against. This advantage is therefore structural rather than evidence of broader robustness. As Table 6.5 shows, PGD-AT loses 5–6 pp to AttackDRO++ on held-out ℓ_2 attacks, while DDN-AT shows the opposite failure mode by collapsing under ℓ_∞ stress. Single-attack training therefore produces specialists; AttackDRO++ does not match PGD-AT's ℓ_∞ depth, but provides the best average cross-attack balance.

6.1.4 Training Stability: How Predictable Is the Outcome?

In addition to improving the mean of average-case robustness metrics, AttackDRO++ also reduces seed-to-seed variation relative to Multi-AT. Table 6.8 compares the standard deviation of each seed-level metric between the two methods.

Table 6.8: Variance reduction across 20 paired random-seed runs. σ is the standard deviation of the per-seed metric. AttackDRO++ substantially reduces seed-to-seed variation on Mean(8), held-out averages, and held-out norm-specific metrics, while Worst(8) and AutoAttack standard deviations remain similar across methods.

Metric	σ Multi-AT (pp)	σ AttackDRO++ (pp)	Ratio
Mean(8)	0.447	0.148	3.03×
Held-out attack mean	0.505	0.155	3.26×
Held-out ℓ_2 mean	0.603	0.223	2.70×
Held-out ℓ_∞ mean	0.427	0.170	2.51×
Seen-source mean	0.303	0.166	1.83×
Worst(8)	0.281	0.237	1.18×
AutoAttack-ℓ_∞	1.167	1.050	1.11×

The reduction is strongest for held-out robustness. The standard deviation decreases by $3.26\times$ for the held-out attack mean, $2.70\times$ for the held-out ℓ_2 mean, and $2.51\times$ for the held-out ℓ_∞ mean. Mean(8) also becomes substantially more stable, decreasing from 0.447 pp under Multi-AT to 0.148 pp under AttackDRO++, a $3.03\times$ reduction.

This suggests that the anchored Cluster-DRO objective not only improves the mean of average-case robustness metrics, but also makes the Multi-AT training outcome less sensitive to random seed. The effect is weaker for Worst(8) and AutoAttack, whose standard deviations remain similar across methods. This is consistent with the previous sections: AttackDRO++ preserves the hardest ℓ_∞ stress metrics relative to Multi-AT, while its main benefit is on average and held-out robustness.

The whitebox results give the main aggregate evidence: AttackDRO++ improves average and held-out robustness relative to Multi-AT while preserving the hardest standardized checks. However, aggregate and per-attack metrics can still hide which CIFAR-10 classes and attack–class cells drive the remaining failures. The next section therefore drills down into class-wise whitebox robustness to test whether the observed gains are broad or concentrated in a small part of the evaluation grid.

6.2 Class-wise Whitebox Robustness Across Attacks

The aggregate results in Section 6.1 show that AttackDRO++ improves average cross-attack robustness while preserving a similar worst-case profile to the uniform multi-attack baseline. However, aggregate accuracy can hide localized failure modes: a method may improve the mean by increasing already-easy cells, while leaving hard class–attack combinations unchanged. To examine whether the improvement is distributed across the evaluation grid, we perform a whitebox per-(attack $\times$ class) drilldown on the eight non-AutoAttack attacks and the ten CIFAR-10 classes. Each cell therefore corresponds to the robust accuracy of one method on one attack and one class, averaged over five seeds.

This analysis has two purposes. First, it identifies whether AttackDRO++ behaves as a balanced multi-attack method or merely shifts robustness from one threat family to another. Second, it exposes the tail cells that dominate practical failure behavior, especially under hard attacks such as PGD-ℓ_∞ and MI-FGSM on visually ambiguous classes. We compare AttackDRO++ against three baselines: the uniform multi-attack model (Multi-AT), the PGD-ℓ_∞ specialist (PGD-AT), and the DDN-ℓ_2 specialist (DDN-AT).

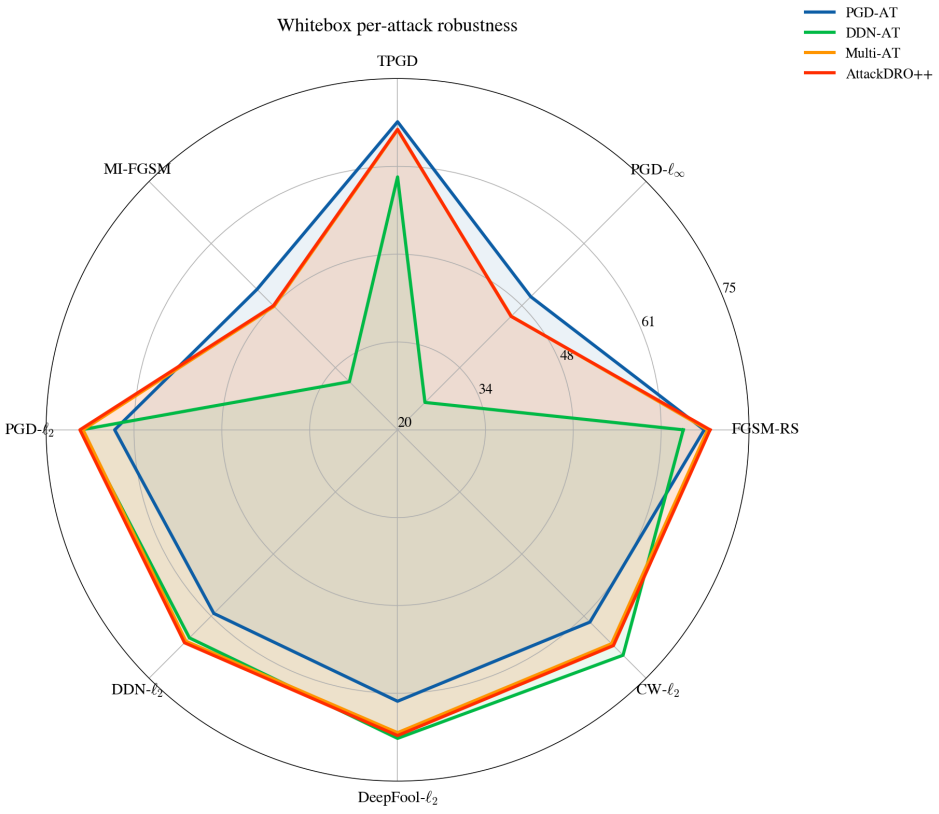


Figure 6.1: Whitebox per-attack robustness profile averaged over five seeds. The radar plot compares PGD-AT, DDN-AT, Multi-AT, and AttackDRO++ (Ours) across the eight non-AutoAttack evaluation attacks. The shape of each curve highlights the specialization of the single-attack baselines and the more balanced profile of the multi-attack methods.

Absolute per-class heatmaps with a shared scale. Before analyzing deltas, we first report the raw per-(class × attack) robust accuracy of each method. All four heatmaps use the same global color scale from 0% to 100%. This shared scale is important because it makes the maps visually comparable across methods: a darker cell always represents the same absolute robust accuracy, regardless of which method is being shown.

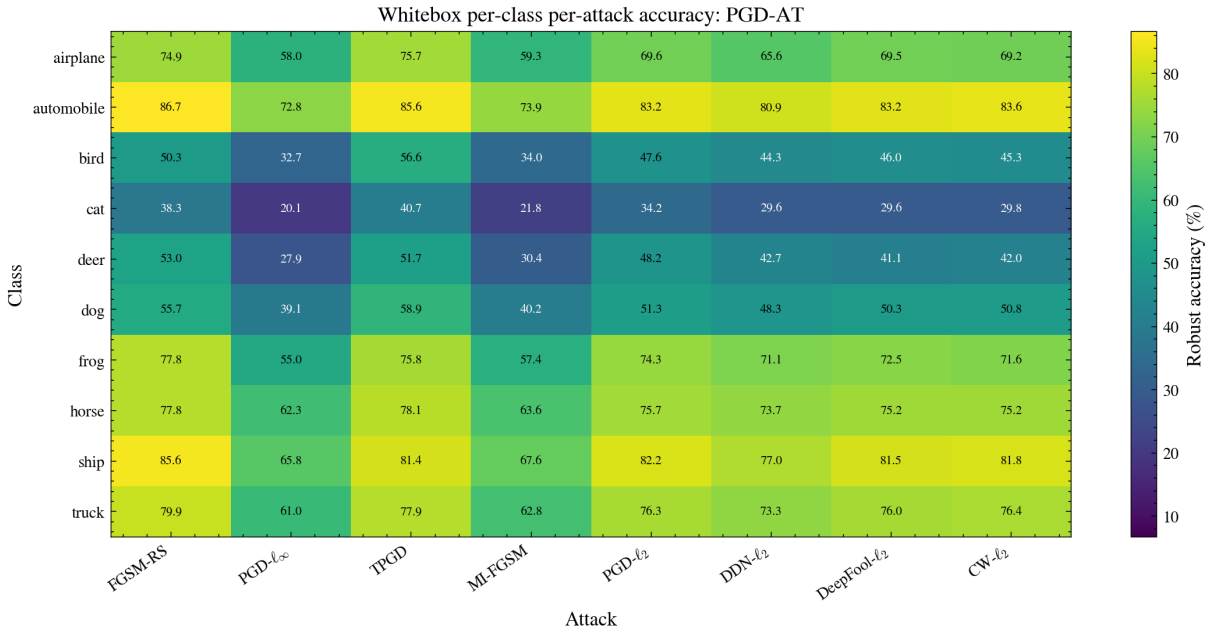


Figure 6.2: Absolute whitebox per-(class × attack) robust accuracy for PGD-AT. Cells report robust accuracy (%).

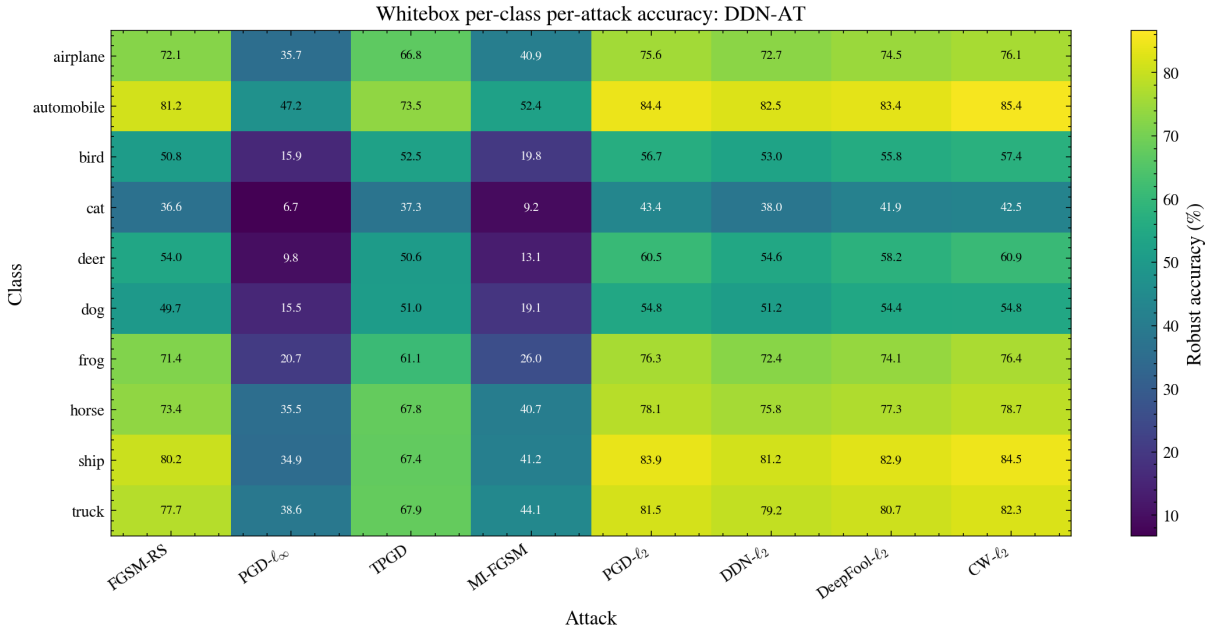


Figure 6.3: Absolute whitebox per-(class × attack) robust accuracy for DDN-AT. Cells report robust accuracy (%).

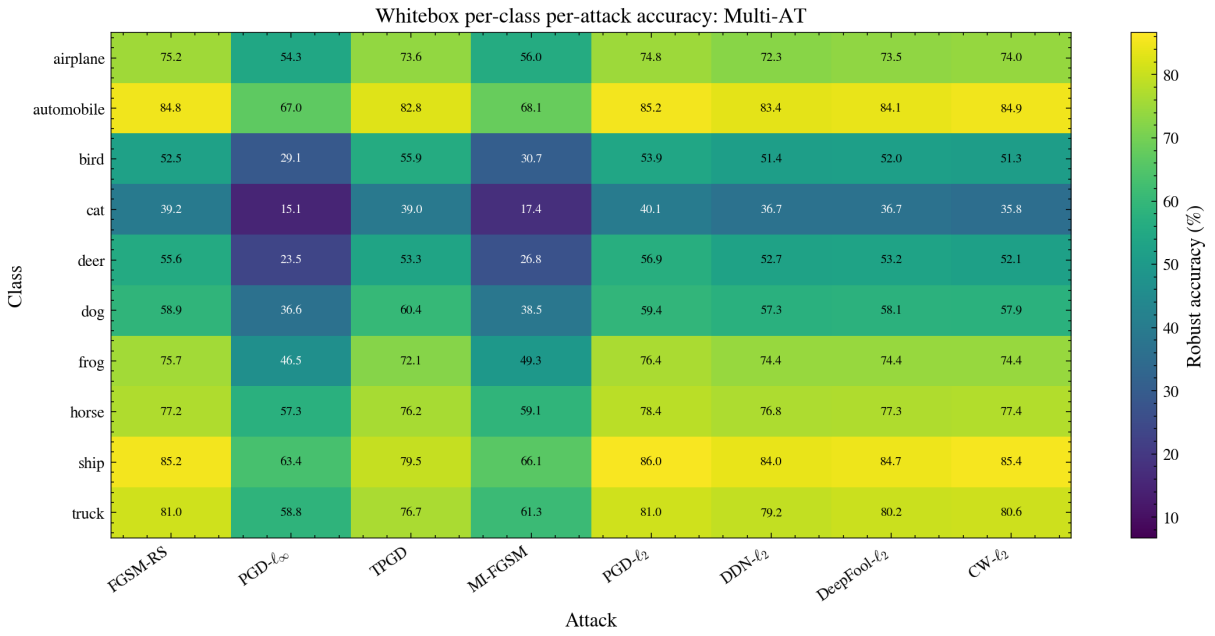


Figure 6.4: Absolute whitebox per-(class $\times$ attack) robust accuracy for Multi-AT. Cells report robust accuracy (%).

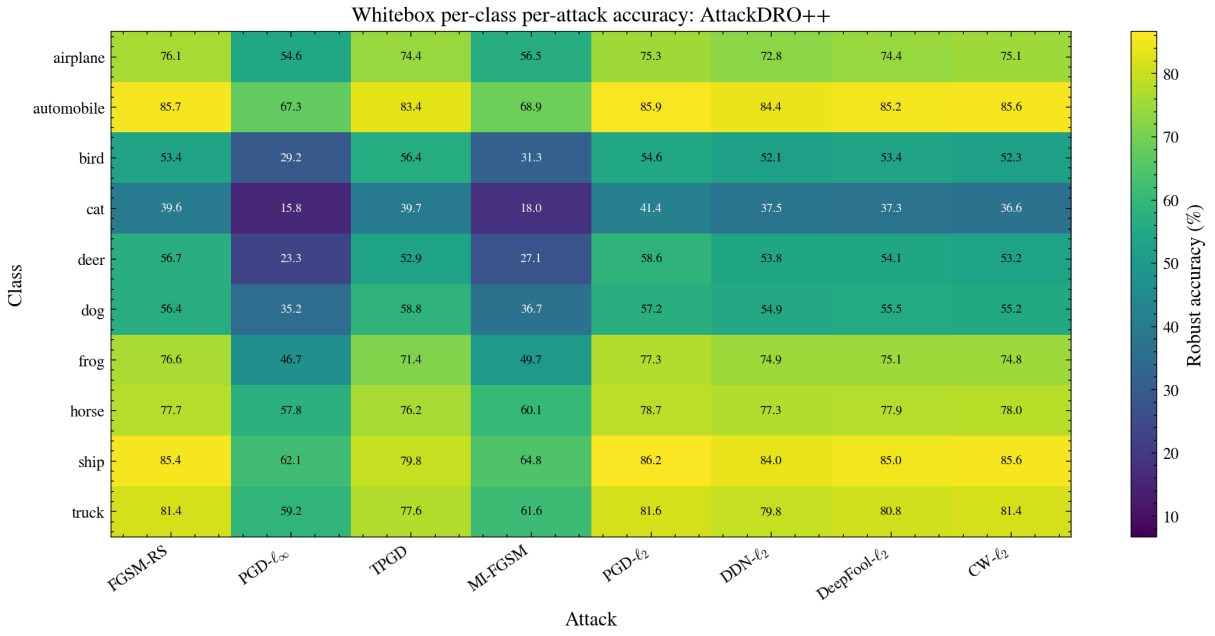


Figure 6.5: Absolute whitebox per-(class $\times$ attack) robust accuracy for AttackDRO++ (Ours). Cells report robust accuracy (%).

PGD-AT is stronger on several ℓ_∞ cells, while DDN-AT is more biased toward bounded ℓ_2 attacks and visibly weaker on PGD-ℓ_∞ and MI-FGSM. Compared with the specialists, Multi-AT and AttackDRO++ show a more balanced cross-attack profile. AttackDRO++ largely preserves the Multi-AT structure while slightly lifting many cells, especially outside the hardest ℓ_∞ animal-class tail.

6.2.1 Class-Level Gains and Persistent Difficulties

Across all methods, the per-class robustness pattern is highly non-uniform. Classes such as *automobile*, *ship*, and *truck* tend to remain easier under most attacks, while *cat*, *deer*, *bird*, and *dog* form the recurring hard tail. This pattern is consistent with the visual similarity and intra-class variation of CIFAR-10: animal classes often share local textures and backgrounds, so small adversarial perturbations can more easily move samples across nearby decision regions.

The hardest whitebox cells are concentrated in the ℓ_∞ iterative attacks. For AttackDRO++, the lowest cells are PGD-ℓ_∞ on *cat* and *deer*, followed by MI-FGSM on the same classes. This indicates that, even though AttackDRO++ improves the average multi-attack profile, the most difficult failure modes are still driven by strong ℓ_∞ optimization on semantically similar animal classes. In contrast, bounded ℓ_2 attacks such as PGD-ℓ_2, DDN-ℓ_2, DeepFool-ℓ_2, and CW-ℓ_2 are generally less damaging for the proposed model than the strongest ℓ_∞ cells.

Attack-Class Delta Against Multi-AT Baseline Compared with Multi-AT, AttackDRO++ produces a broad but modest shift rather than a large change concentrated in a few cells. The mean per-cell gain is only $+0.281$ percentage points, but 67 out of 80 attack–class cells are positive.

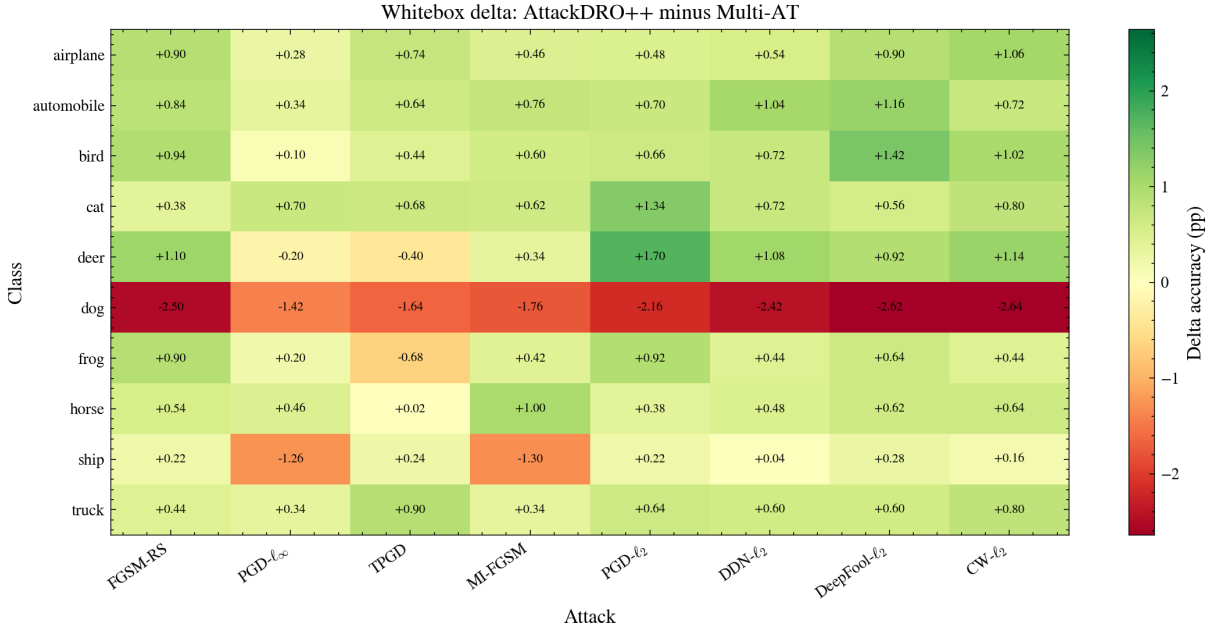


Figure 6.6: Whitebox per-(class $\times$ attack) delta of AttackDRO++ relative to Multi-AT. Each cell reports AttackDRO++ minus Multi-AT in percentage points, averaged over five seeds.

Attack-Class Delta Against Single-AT Baselines Table 6.9 summarizes the per-cell delta between AttackDRO++ and each baseline. The comparison against Multi-AT is the most important one because both methods train on the same source attacks and differ mainly in the group structure used for optimization. AttackDRO++ improves 67/80 cells over Multi-AT, with a mean gain of $+0.281$ percentage points and a bottom-10 lift of $+0.282$ percentage points. The gains are small, but they are widespread, which supports the interpretation that cluster-based DRO acts as a stabilizing refinement over uniform multi-attack training.

Table 6.9: Per-(class $\times$ attack) whitebox comparison of AttackDRO++ against each baseline. Δ is computed as AttackDRO++ minus the baseline. Positive cells count the number of attack–class cells where $\Delta > 0$ out of 80 total cells.

Baseline	Positive cells ($\Delta > 0$)	Mean Δ (pp)	Significant cells AttackDRO++ / Baseline	Bottom-10 lift (pp)	Main pattern
Multi-AT	67/80	$+0.281$	0 / 2	$+0.282$	Balanced refinement
PGD-AT	47/80	$+1.782$	40 / 19	$+0.726$	Gains mainly on ℓ_2 cells
DDN-AT	62/80	$+5.781$	44 / 5	$+15.710$	Gains mainly on ℓ_∞ cells

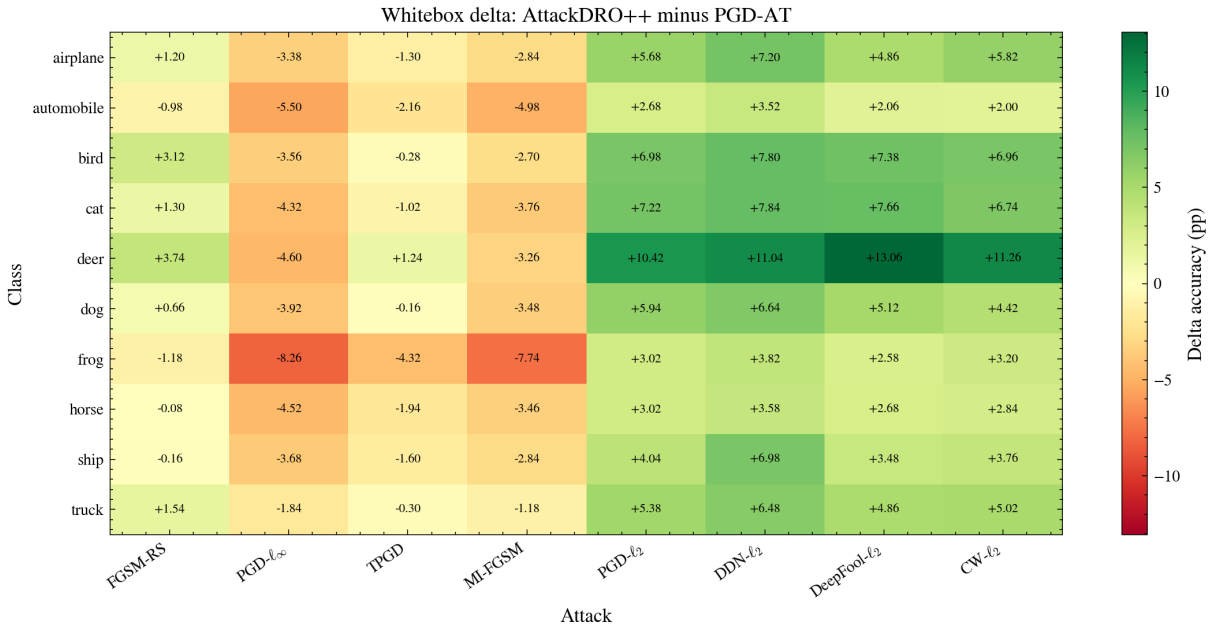


Figure 6.7: Whitebox per-(class $\times$ attack) delta of AttackDRO++ (Ours) relative to PGD-AT. Positive cells indicate where AttackDRO++ (Ours) improves over the PGD-ℓ_∞ specialist. Gains are concentrated mainly on bounded ℓ_2 attacks, while PGD-AT remains stronger on several ℓ_∞ iterative cells.

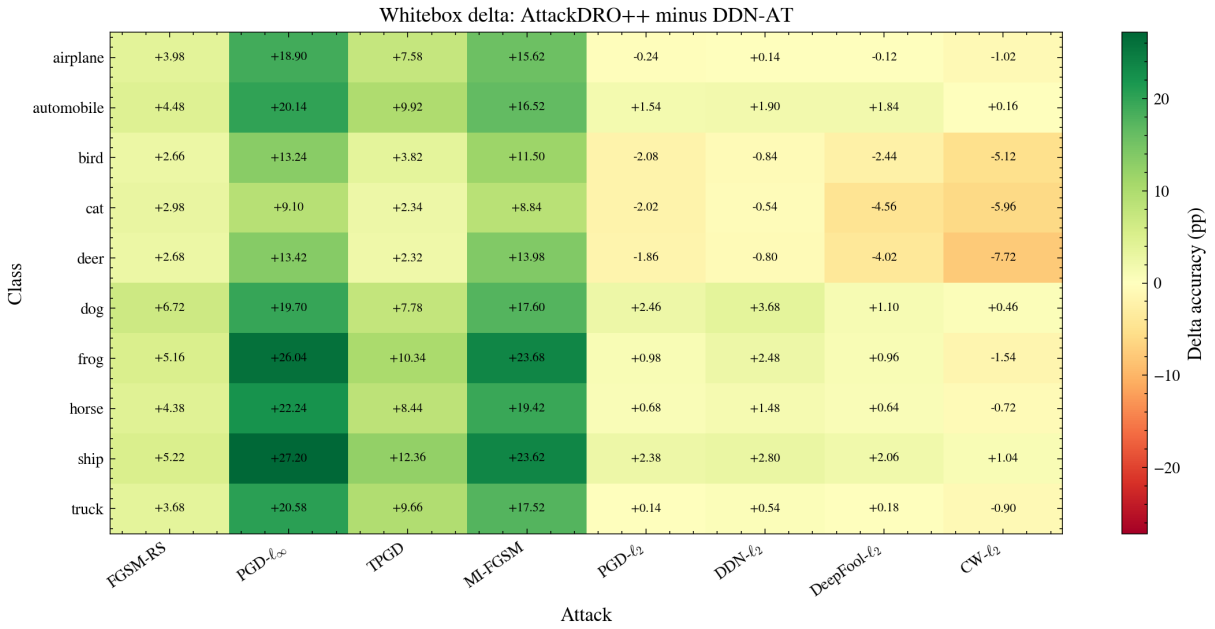


Figure 6.8: Whitebox per-(class × attack) delta of AttackDRO++ (Ours) relative to DDN-AT. The large positive cells under PGD-ℓ_∞ and MI-FGSM show that AttackDRO++ (Ours) substantially corrects the ℓ_∞ weakness of the DDN-ℓ_2 specialist, while several negative cells remain on DDN-AT's preferred ℓ_2 family.

The comparison against PGD-AT shows the expected specialist trade-off. AttackDRO++ is positive in 47/80 cells and has a mean gain of $+1.782$ percentage points, but the gains and losses are not uniformly distributed. The largest improvements occur on bounded ℓ_2 attacks, especially DeepFool-ℓ_2, CW-ℓ_2, DDN-ℓ_2, and PGD-ℓ_2 on hard animal classes such as *deer*, *cat*, and *bird*. For example, the largest gains over PGD-AT appear on DeepFool-ℓ_2 for *deer* ($+13.06$ pp), CW-ℓ_2 for *deer* ($+11.26$ pp), and DDN-ℓ_2 for *deer* ($+11.04$ pp). Conversely, PGD-AT remains stronger on several ℓ_∞ cells, particularly PGD-ℓ_∞ and MI-FGSM for *frog*, *automobile*, *horse*, and *cat*. This confirms that PGD-AT is not weak in absolute terms; rather, it is specialized toward the ℓ_∞ geometry used during training.

The comparison against DDN-AT is even more asymmetric. AttackDRO++ improves 62/80 cells and gains $+5.781$ percentage points on average, with a large $+15.710$ percentage-point improvement on the bottom-10 cells. The largest improvements occur under ℓ_∞ attacks: PGD-ℓ_∞ on *ship* ($+27.20$ pp), PGD-ℓ_∞ on *frog* ($+26.04$ pp), MI-FGSM on *frog* ($+23.68$ pp), and MI-FGSM on *ship* ($+23.62$ pp). These large deltas show that DDN-AT develops a strong ℓ_2 bias and leaves many ℓ_∞ attack–class cells under-protected. At the same time, DDN-AT remains stronger on some ℓ_2 boundary/optimization cells, especially CW-ℓ_2 and DeepFool-ℓ_2 for *deer*, *cat*, and *bird*. Thus, AttackDRO++ improves cross-attack balance but does not completely dominate the

strongest single-norm specialist on its preferred family.

Attack-family interpretation. The attack-wise deltas make the specialization effect clear. Relative to PGD-AT, AttackDRO++ loses on PGD-ℓ_∞ and MI-FGSM on average, but gains strongly on the bounded ℓ_2 attacks. Relative to DDN-AT, the pattern reverses: AttackDRO++ gains substantially on PGD-ℓ_∞, MI-FGSM, TPGD, and FGSM-RS, while being slightly weaker on CW-ℓ_2 and DeepFool-ℓ_2. The Multi-AT comparison is more balanced: most attack averages are positive, but the magnitudes are below one percentage point.

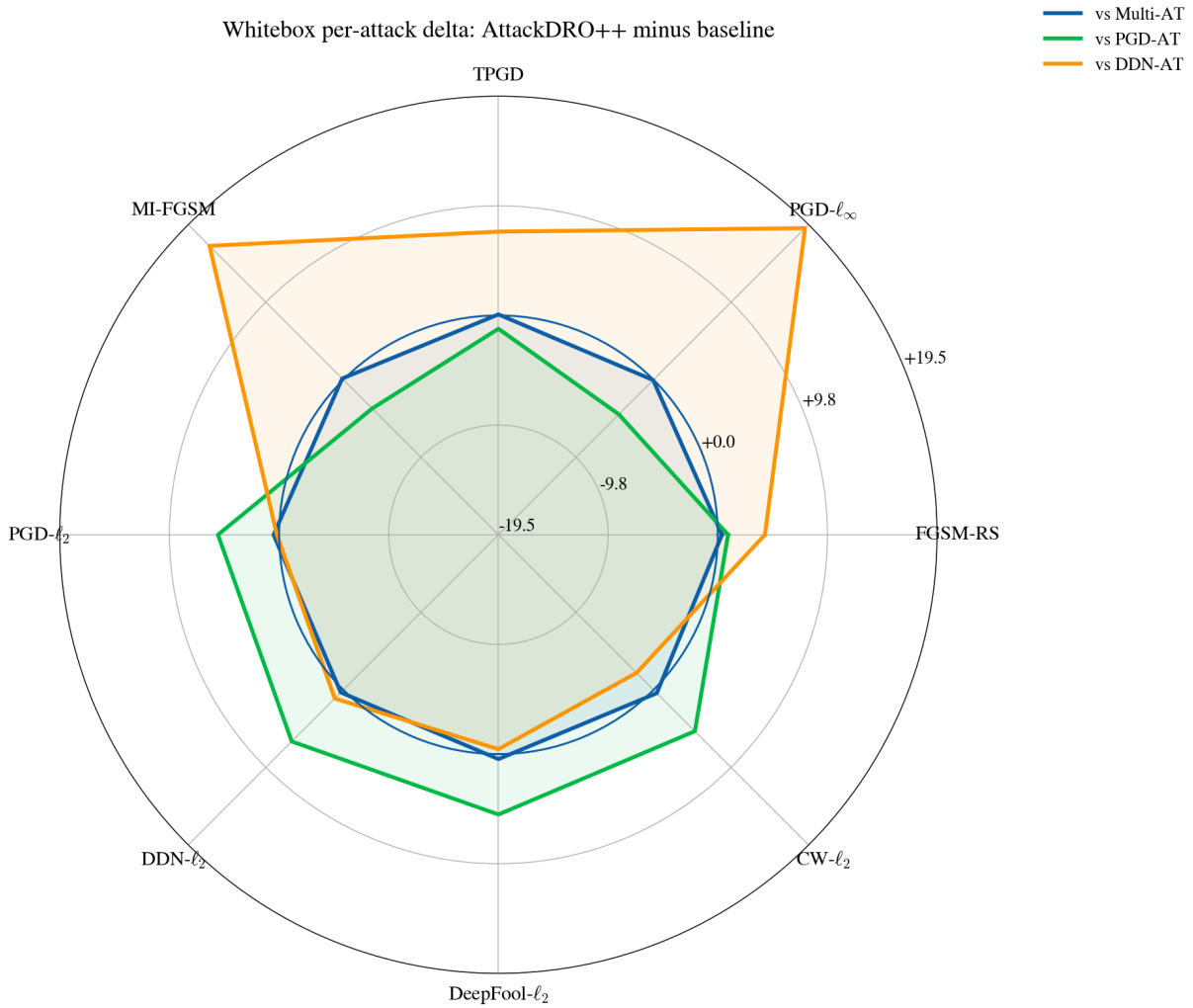


Figure 6.9: Attack-wise whitebox delta of AttackDRO++ relative to the three baselines.

This result supports the main claim of the chapter. AttackDRO++ should not be interpreted as a replacement for a single specialist when the deployment threat model is known exactly. Instead, it is a cross-attack robustness method: it sacrifices some of the peak strength of a specialist in exchange for a more even robustness profile across attacks and classes.

6.2.2 Tail-Class Robustness: Do the Weakest Cells Improve?

We next examine the weakest attack–class cells for each baseline, using $K \in \{5, 10, 20\}$. These cells capture localized failure modes that can be hidden by mean accuracy. For Multi-AT, the weakest cells are mainly strong ℓ_∞ attacks, especially PGD-ℓ_∞ and MI-FGSM, on hard animal classes such as *cat*, *deer*, and *bird*. AttackDRO++ improves this tail only slightly: the bottom-10 lift is $+0.282$ pp. Although small, this shows that the average gain over Multi-AT is not obtained by sacrificing its weakest cells.

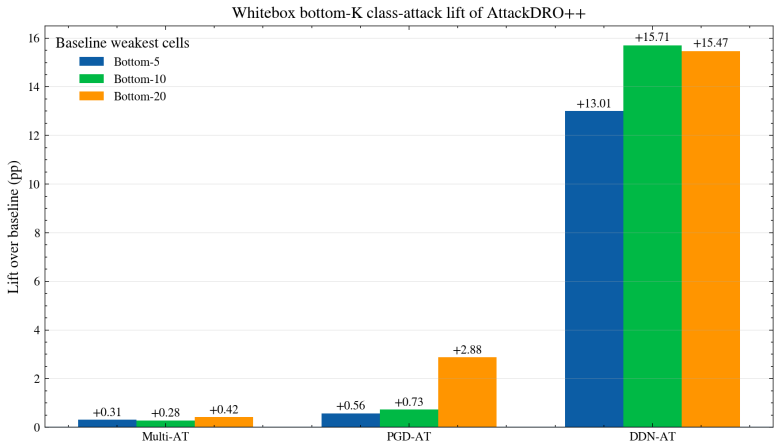


Figure 6.10: Weakest-K whitebox class–attack lift of AttackDRO++ over each baseline. For each baseline, the weakest-K cells are the lowest-accuracy class–attack cells of that baseline; bars report AttackDRO++ minus the baseline on those same cells.

The tail comparison is most pronounced against DDN-AT. Its weakest cells collapse under ℓ_∞-oriented attacks, with the bottom-10 cells averaging only 15.602%. AttackDRO++ raises these same cells by $+15.710$ pp, explaining why it achieves a much more balanced whitebox profile than the ℓ_2 specialist, even though DDN-AT remains competitive on several ℓ_2 cells.

Against PGD-AT, the lift is smaller but still positive: $+0.726$ pp on the bottom-10 cells. PGD-AT's weakest cells include several bounded ℓ_2 attacks on difficult classes such as *cat* and *deer*; AttackDRO++ reduces many of these failures, indicating that multi-attack training and the Cluster-DRO component help mitigate the cross-norm weakness of PGD-only training.

Overall, the weakest-cell analysis supports the aggregate conclusion from Section 6.1: AttackDRO++ gives a broad but modest refinement over Multi-AT, substantially corrects the ℓ_∞ tail weakness of DDN-AT, and reduces the ℓ_2 tail weakness of PGD-AT.

The class-wise whitebox analysis shows where the aggregate gains appear and which hard cells remain. It still evaluates attacks generated directly against the target check-

point, so it does not answer whether the same robustness pattern holds when adversarial examples are crafted on a different model. The next section therefore moves to the gray-box transfer protocol from Section 5.8.

6.3 Graybox Robustness Across Surrogate Models

The whitebox results in Section 6.1 show that AttackDRO++ improves average and held-out robustness over Multi-AT on $n = 20$ paired seeds. We next test whether this gain also holds under cross-model transfer, using the graybox protocol of Section 5.8. In this setting, adversarial examples are generated on one checkpoint and evaluated on an independently trained target checkpoint.

Random-seed checkpoint panel and whitebox sanity check. The graybox analysis uses 20 random-seed checkpoints to define the surrogate–target evaluation panel. These checkpoints come from four method families evaluated across five random seeds each and are fixed before graybox transfer is analyzed. The diagonal sanity check below compares this random-seed checkpoint panel against the full whitebox panel to verify that the main method ordering is preserved.

The diagonal entries of the graybox table correspond to the case where the surrogate and target are the same checkpoint. We use these entries as a whitebox sanity check. They need not match the main whitebox evaluation exactly, because the graybox cache uses a balanced 125-image-per-class-per-attack split instead of the full test set for each attack. However, they should preserve the same method ordering and remain close in absolute value.

Table 6.10: Whitebox sanity check from the graybox pipeline. The five-random-seed columns use diagonal graybox entries, where surrogate and target are the same checkpoint. The 20-random-seed columns report the main whitebox results from Table 6.1. Bold values indicate the best method for each metric.

Target method	Mean(8) $n = 5$	Worst(8) $n = 5$	Mean(8) $n = 20$	Worst(8) $n = 20$
PGD-AT	61.524	**50.384**	60.499	**49.656**
DDN-AT	56.922	25.568	56.226	25.957
Multi-AT	62.458	45.584	62.045	45.082
AttackDRO++ (Ours)	**62.642**	45.088	**62.324**	45.079

The Mean(8) ordering is identical on both panels — AttackDRO++ $>$ Multi-AT $>$ PGD-AT $>$ DDN-AT — and absolute values agree to within 1.0 pp. More importantly, the paired Mean(8) delta of AttackDRO++ minus Multi-AT is $+0.184$ pp on the $n = 5$ graybox panel and $+0.279$ pp on the full $n = 20$ panel: both are positive and of the same order of magnitude. The graybox cache is therefore consistent with the main evaluation

protocol and supports the cross-method analyses below.

6.3.1 Cross-Method Transfer Structure

We aggregate the per-attack per-class transfer table into a method-level 4×4 transfer matrix. Rows index the target method being evaluated, and columns index the surrogate method that generates adversarial examples. Each cell reports the target robust accuracy averaged over all matching target–surrogate seed pairs and the eight evaluation attacks. Higher values indicate that the row target is more robust to adversarial examples crafted on the column surrogate.

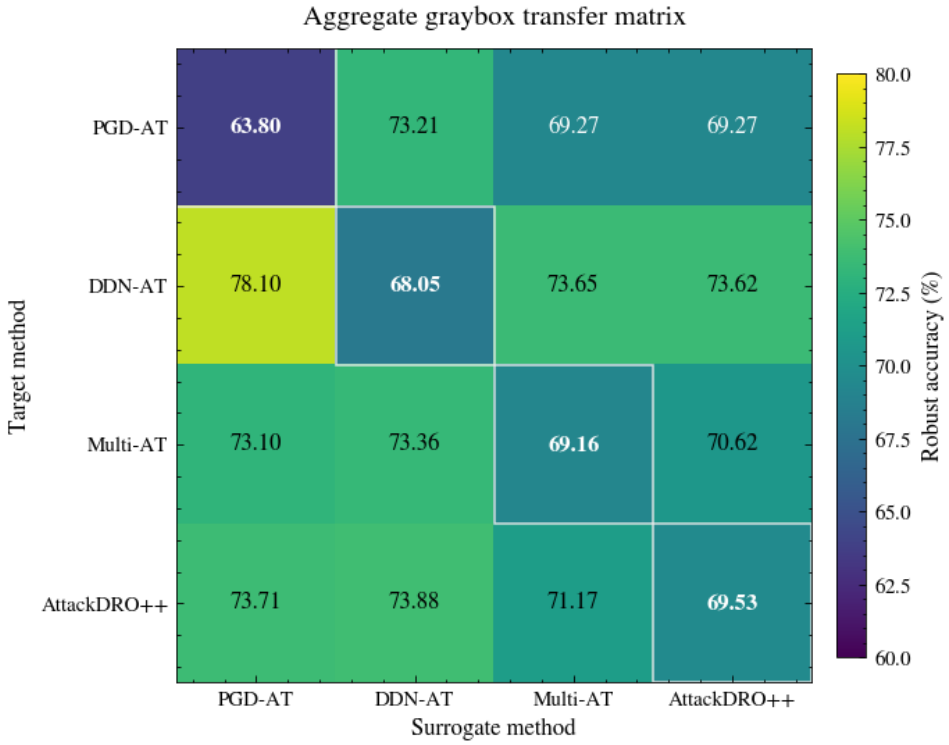
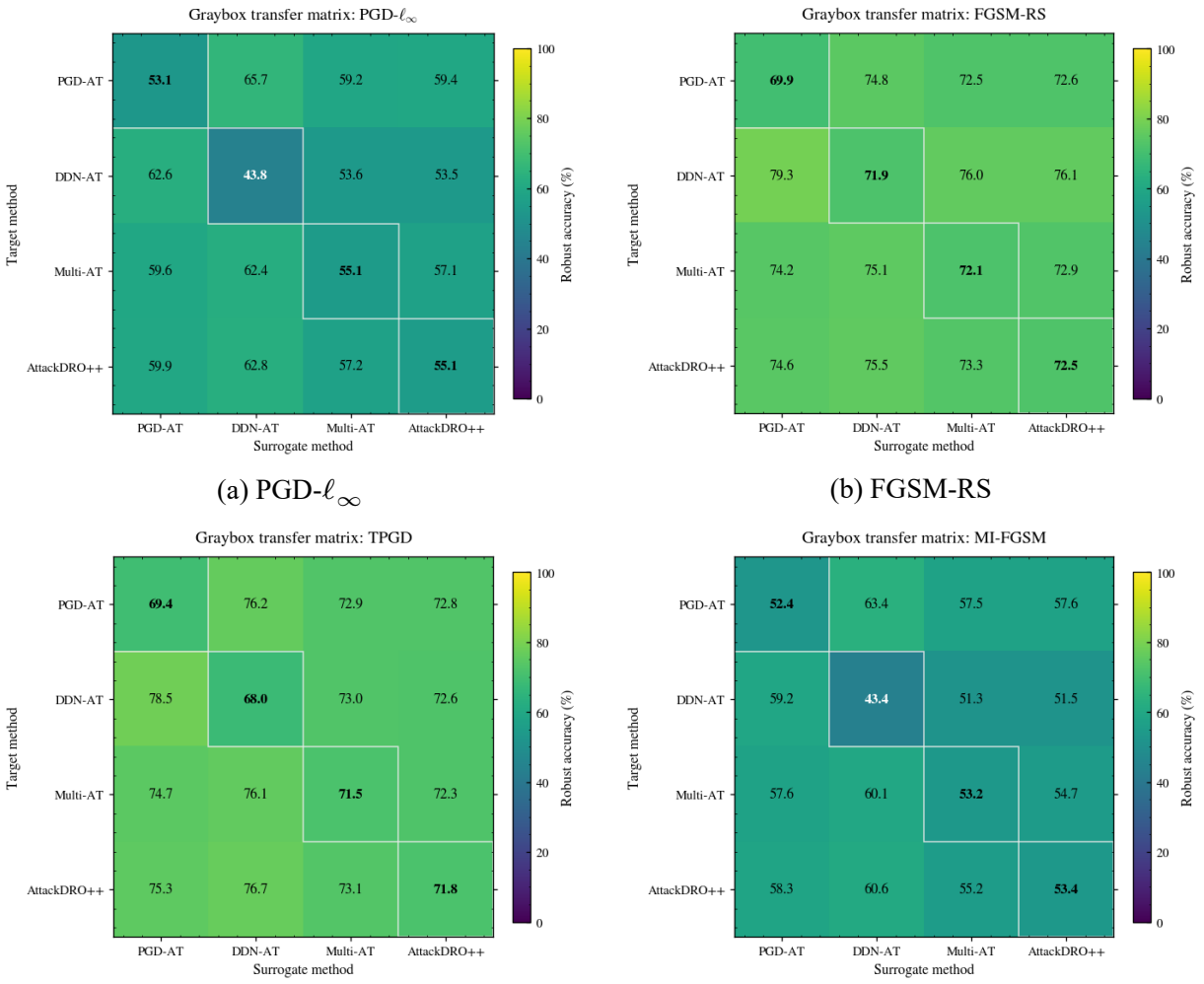


Figure 6.11: Method-level graybox transfer matrix on the $n = 5$ panel. Rows are target methods and columns are surrogate methods. Cell values are robust accuracy (%) averaged over seeds and the eight evaluation attacks.

Two structural patterns are visible. First, rows ranked by mean off-diagonal accuracy give DDN-AT (75.12%) > AttackDRO++ (72.92%) > Multi-AT (72.36%) > PGD-AT (70.59%). The DDN-AT row is highest not because DDN-AT is the strongest defense — its whitebox Mean(8) is the lowest of the four — but because ℓ_∞ adversarial examples generated by other surrogates transfer poorly to a DDN-AT target whose decision boundary is shaped by ℓ_2 training. Section 6.3.2 quantifies this structural inflation explicitly through the whitebox–graybox gap. Second, the AttackDRO++ row is consistently above the Multi-AT row for every surrogate, with cross-method gaps ranging from $+0.17$ pp (DDN-AT surrogate) to $+0.61$ pp (PGD-AT surrogate). The whitebox advan-

tage of AttackDRO++ over Multi-AT therefore persists when adversarial examples are crafted on independently trained surrogates.

The aggregate transfer matrix hides important attack-family structure. Figures 6.12 and 6.13 therefore disaggregate the transfer behavior into the four ℓ_∞-oriented attacks and the four ℓ_2-oriented attacks. This grouping makes the specialist behavior more explicit.



(a) PGD-ℓ_∞

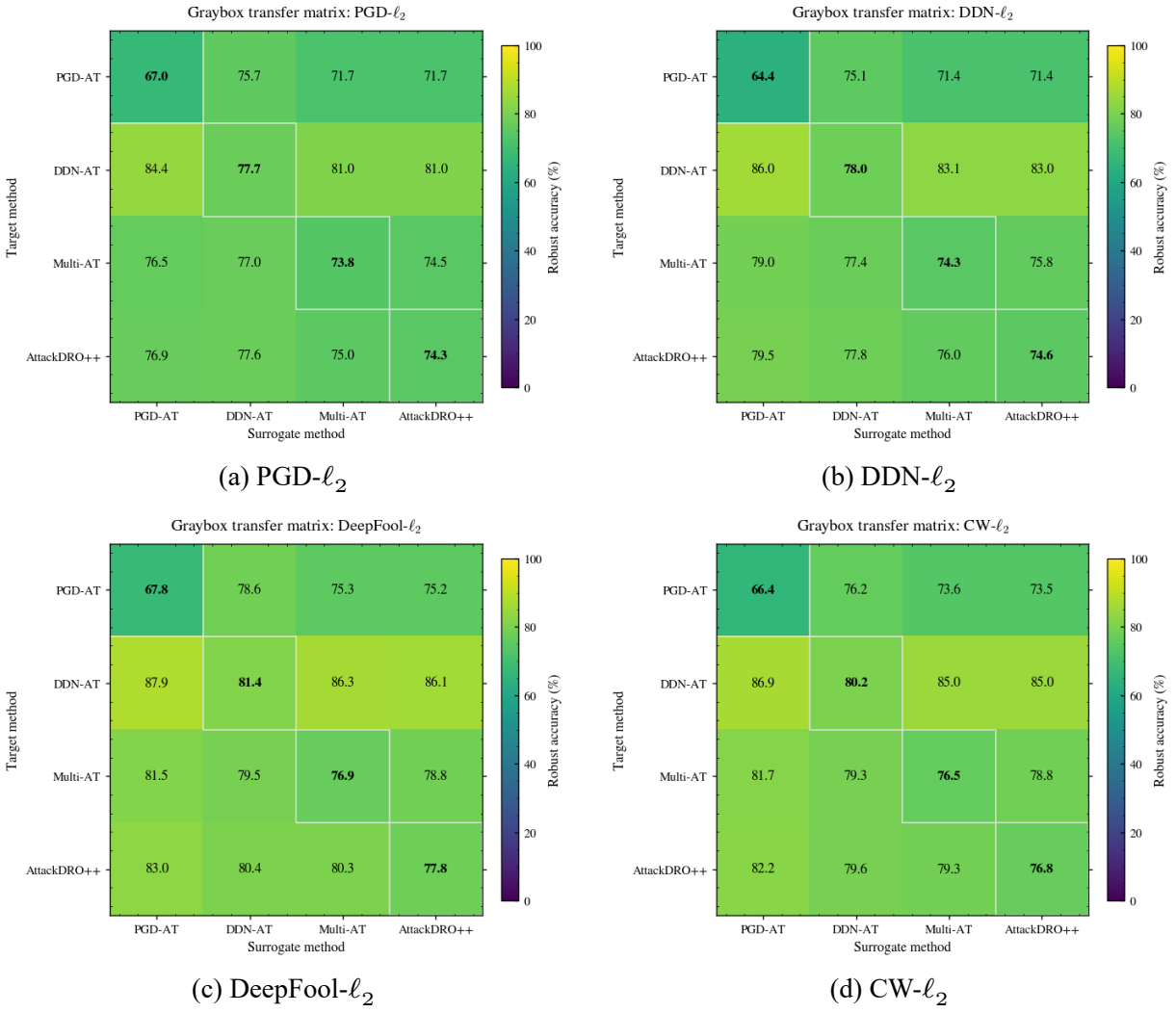
(b) FGSM-RS

(c) TPGD

(d) MI-FGSM

Figure 6.12: Per-attack graybox transfer matrices for the ℓ_∞-oriented attacks. Rows are target methods and columns are surrogate methods; cells report robust accuracy (%).

On the ℓ_∞ attacks, this advantage disappears: PGD-AT and AttackDRO++ are more competitive, and AttackDRO++ remains closer to the balanced Multi-AT profile. Thus, the per-attack matrices show that DDN-AT's strong graybox row average should be interpreted as family-specific transfer specialization rather than uniform robustness.



(a) PGD-ℓ_2 (b) DDN-ℓ_2

(c) DeepFool-ℓ_2 (d) CW-ℓ_2

Figure 6.13: Per-attack graybox transfer matrices for the ℓ_2-oriented attacks. Rows are target methods and columns are surrogate methods; cells report robust accuracy (%).

On the ℓ_2 attacks, the DDN-AT target row is consistently strong, especially when attacks are transferred from PGD-AT, Multi-AT, or AttackDRO++ surrogates. This indicates that DDN-AT's high aggregate graybox score is driven mainly by ℓ_2-family transfer behavior, rather than by uniformly strong robustness across both attack geometries. AttackDRO++ remains closer to the balanced Multi-AT profile, while still improving over Multi-AT on most ℓ_2-oriented transfer cells.

6.3.2 Paired Graybox Comparisons: Does the Whitebox Advantage Carry Over?

The main graybox comparison is between AttackDRO++ and Multi-AT. Both methods train on the same source attacks $\{\text{PGD-}\ell_\infty,\ \text{DDN-}\ell_2\}$ and use the same backbone and schedule; they differ only in the use of latent clustering, gradient fingerprints, and the anchored Cluster-DRO objective.

We use the symmetric paired test that has already been described in Section 5.8: for every triple $(\text{target seed}, \text{surrogate seed}, \text{attack})$, we pair the configuration "AttackDRO++ as target, Multi-AT as surrogate" against its mirror "Multi-AT as target, AttackDRO++ as surrogate" and compute the per-triple difference. As the results, $n = 5 \times 5 \times 8 = 200$ paired pairs make the comparison invariant to which method is chosen as the surrogate.

Table 6.11: Paired graybox comparisons. Each row uses $n = 200$ paired target–surrogate–attack triples. Positive Δ favors the reference method named in the panel heading.

Compared method	Accuracy		Δ	d_z	t	p
	Reference	Baseline	(pp)			
AttackDRO++ (Ours) vs. baselines						
Multi-AT	71.17	70.62	+0.549	0.42	5.96	1.1×10^{-8}
PGD-AT	73.71	69.27	+4.435	1.33	18.78	6.1×10^{-46}
DDN-AT	73.88	73.62	+0.258	0.04	0.60	0.549
Multi-AT vs. single-attack baselines						
PGD-AT	73.10	69.27	+3.829	1.19	16.89	2.7×10^{-40}
DDN-AT	73.36	73.65	-0.292	-0.05	-0.67	0.504

The AttackDRO++ vs. Multi-AT contrast is the headline result of this section: under symmetric cross-method graybox transfer, AttackDRO++ improves onMulti-AT by $+0.549$ pp at $p \approx 1.1 \times 10^{-8}$, with a small-to-medium paired effect size of $d_z \approx 0.42$. This effect is roughly twice the corresponding whitebox delta of $+0.279$ pp on the same seed panel and is significant at far stricter levels. The Cluster-DRO improvement therefore *strengthens* rather than disappears under graybox transfer, indicating that the gain originates from a broader cross-attack robustness profile rather than from a defense narrowly tuned to AttackDRO++'s own gradients.

The single-attack contrasts confirm the expected specialist–versus–balanced pattern. AttackDRO++ improves on PGD-AT by $+4.44$ pp at $p < 10^{-45}$ ($d_z = 1.33$, a large effect), tracking the whitebox trend that PGD-AT loses heavily on the ℓ_2 family it does not train against. The AttackDRO++ vs. DDN-AT contrast is statistically indistinguishable from zero ($+0.26$ pp, $p = 0.55$); this reflects the DDN-AT graybox inflation discussed above, not actual parity. We dissect this asymmetry in Section 6.3.2.

Per-attack transfer behavior. To check whether the $+0.549$ pp Mean(8) gain over Multi-AT is broad or concentrated, we report paired graybox deltas attack by attack on the same $n = 200$ pair set. Each attack contributes 25 paired triples (one per surrogate–target seed combination).

Table 6.12: Per-attack symmetric graybox comparison between AttackDRO++ and Multi-AT. Each attack uses $n = 25$ paired seed combinations. Significance is reported in the p-value column.

Attack	Family	A mean (%)	B mean (%)	Δ (pp)	p
FGSM-RS	held-out ℓ_∞	73.32	72.86	$+0.454$	0.0114
PGD-ℓ_∞	seen ℓ_∞	57.17	57.07	$+0.102$	0.307
TPGD	held-out ℓ_∞	73.09	72.30	$+0.790$	0.0240
MI-FGSM	held-out ℓ_∞	55.20	54.75	$+0.458$	0.0022
PGD-ℓ_2	held-out ℓ_2	74.97	74.53	$+0.432$	0.0373
DDN-ℓ_2	seen ℓ_2	76.04	75.79	$+0.256$	0.341
DeepFool-ℓ_2	held-out ℓ_2	80.29	78.82	$+1.469$	0.0002
CW-ℓ_2	held-out ℓ_2	79.26	78.83	$+0.429$	0.249

AttackDRO++ produces a positive paired delta on all eight attacks. Five of these deltas are individually significant at $p < 0.05$, and the gain is largest on DeepFool-ℓ_2 ($+1.47$ pp, $p = 1.6 \times 10^{-4}$), the most local-gradient ℓ_2 attack and the held-out attack on which the whitebox gain over Multi-AT was also largest. The two seen attacks (PGD-ℓ_∞ and DDN-ℓ_2) contribute small, non-significant gains, consistent with the fact that both methods already train against these attacks. The graybox Mean(8) gain is therefore distributed across attack families rather than concentrated on one family.

We aggregate the per-attack deltas into the four attack groups defined in Section 5.7 to compare the cross-method graybox profile across all four methods.

Table 6.13: Cross-method graybox accuracy by attack family. Values are off-diagonal target accuracy (%) averaged over the $n = 5$ panel. Bold values indicate the best method for each metric.

Target method	Seen ℓ_∞	Seen ℓ_2	Heldout ℓ_∞	Heldout ℓ_2	Mean(8)
PGD-AT	**61.48**	72.61	68.93	74.60	70.59
DDN-AT	56.58	**84.03**	68.60	**84.85**	**75.12**
Multi-AT	59.71	77.40	68.64	78.62	72.36
AttackDRO++ (Ours)	59.96	77.78	**69.18**	79.35	72.92

The family decomposition makes three points concrete. The held-out ℓ_∞ column has AttackDRO++ at the top (69.18%) by a small but consistent margin over the other three methods. The held-out ℓ_2 column is dominated by DDN-AT (84.85%), with AttackDRO++ (79.35%) ranked second and improving on Multi-AT by $+0.73$ pp. The seen ℓ_2 column shows the same pattern. The seen ℓ_∞ column favors PGD-AT, again by structure: surrogates generate strong PGD-ℓ_∞ adversarial examples, and only PGD-AT is robust to them. AttackDRO++ is the only method that is at or above Multi-AT on every attack family, while DDN-AT and PGD-AT each remain narrow specialists under transfer.

Whitebox–graybox gap and specialist inflation. The DDN-AT target row in Figure 6.11 and the DDN-AT row in Table 6.13 are the highest under graybox transfer, even though DDN-AT has the lowest whitebox Mean(8) on both the $n = 5$ and the full $n = 20$ panels. We resolve this apparent contradiction by quantifying the gap between whitebox and cross-method graybox accuracy for each method. A large positive gap indicates that a defense looks much stronger under graybox transfer than under direct whitebox attack, which is a signature of a structural transfer mismatch rather than a broader robustness profile.

Table 6.14: Whitebox–graybox gap by target method on the $n = 5$ panel. The gap is cross-method graybox Mean(8) minus diagonal whitebox Mean(8).

Target method	Whitebox Mean(8)	Graybox Mean(8)	Gap (pp)
PGD-AT	61.52	70.59	+ 9.06
DDN-AT	56.92	75.12	+18.20
Multi-AT	62.46	72.36	+ 9.90
AttackDRO++ (Ours)	62.64	72.92	+10.27

PGD-AT, Multi-AT, and AttackDRO++ all show a comparable whitebox–graybox gap of approximately 9–10 pp, indicating that adversarial examples crafted on alternate surrogates are weaker than those crafted directly on the target by a roughly constant amount. DDN-AT, by contrast, gains $+18.20$ pp under graybox transfer, almost twice the gap of any other method. Its row-leading graybox accuracy is therefore not evidence of a stronger defense; it is an artifact of how poorly ℓ_∞ adversarial examples generated by PGD-AT, Multi-AT, and AttackDRO++ surrogates transfer to a purely-ℓ_2-trained target. AttackDRO++ retains the highest whitebox Mean(8) of the four methods while exhibiting a graybox gap that is only $+0.37$ pp larger than that of Multi-AT — consistent with a balanced cross-attack defense rather than a transfer-favored specialist.

The per-(class $\times$ attack) whitebox–graybox gap heatmaps in Figures 6.14–6.17 make the specialist-inflation effect concrete. Cell values are computed as graybox accuracy minus whitebox accuracy, so positive values indicate that transferred attacks are weaker than direct whitebox attacks on the same target.

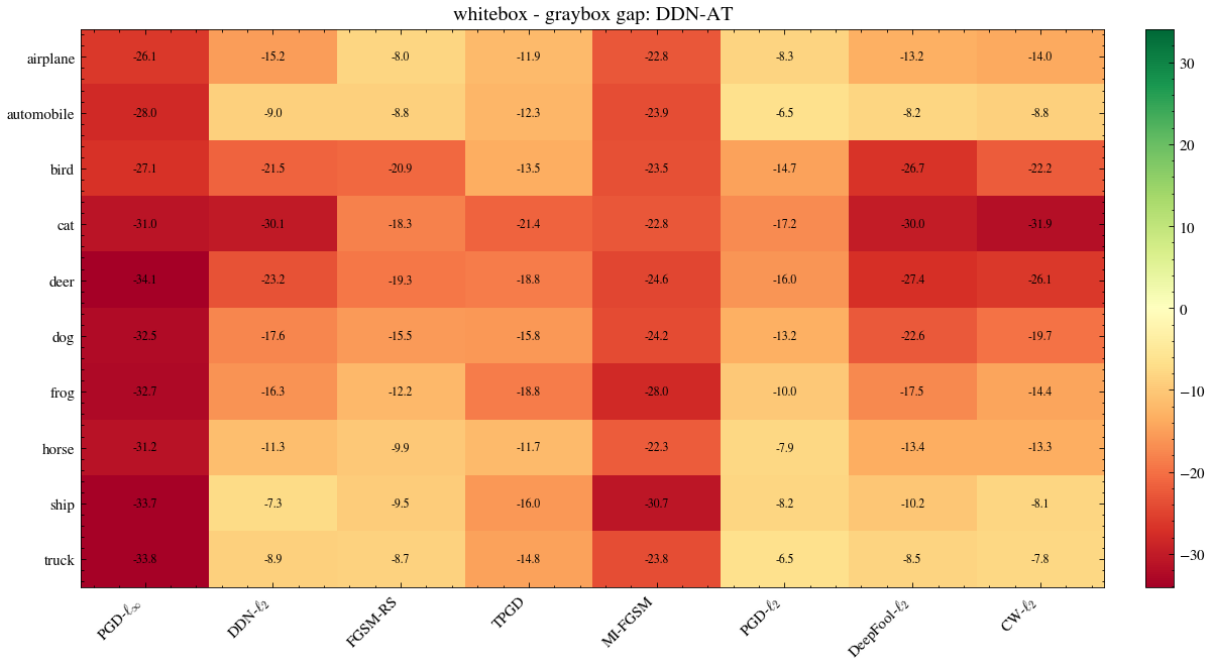


Figure 6.14: Whitebox–graybox gap heatmap for DDN-AT.

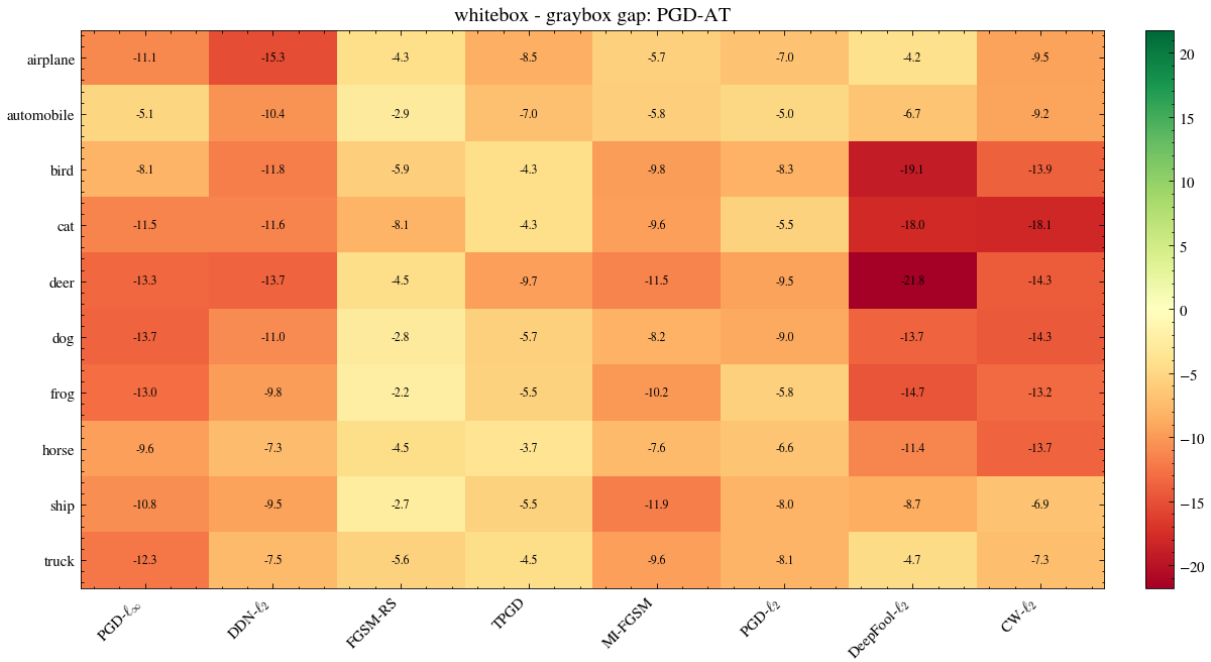


Figure 6.15: Whitebox–graybox gap heatmap for PGD-AT.

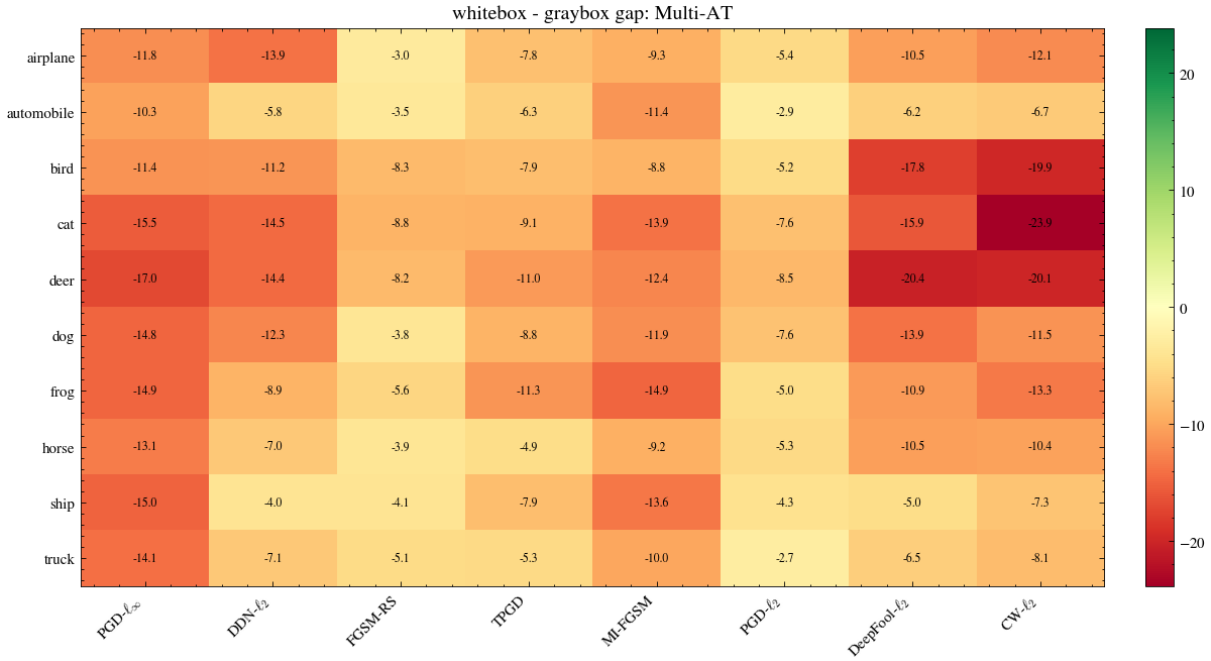


Figure 6.16: Whitebox–graybox gap heatmap for Multi-AT.

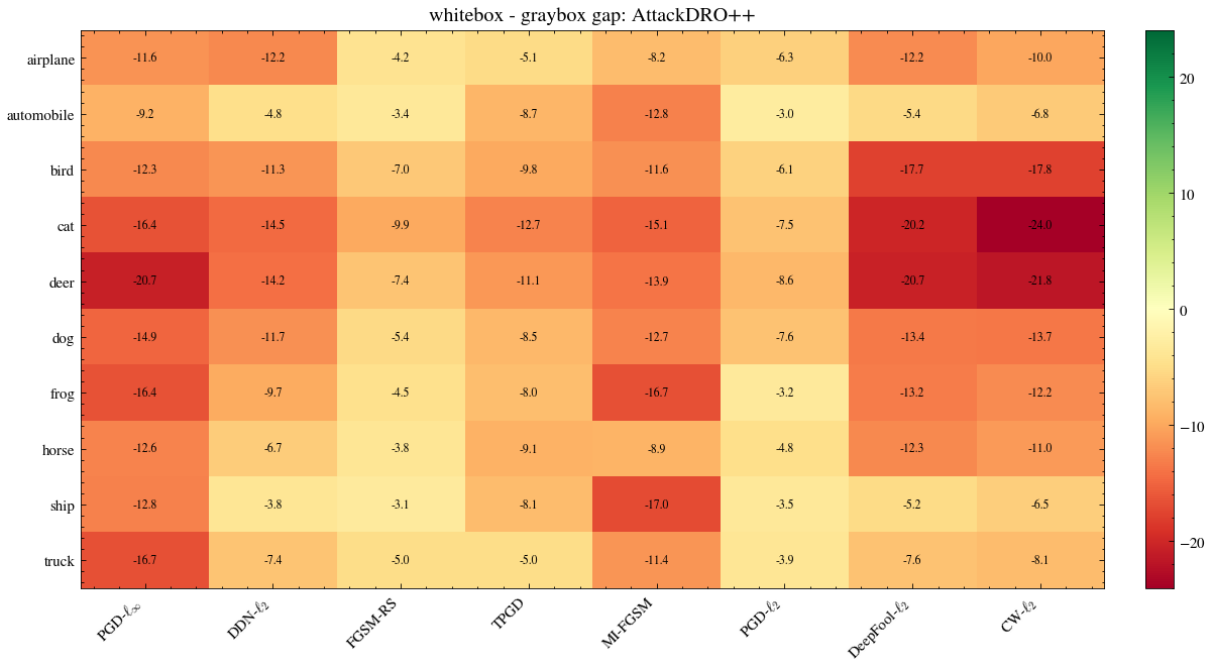


Figure 6.17: Whitebox–graybox gap heatmap for AttackDRO++ (Ours).

DDN-AT exhibits broad positive gaps on the ℓ_∞-oriented attack columns, indicating that its high graybox score is partly driven by weak transfer from ℓ_∞ surrogates rather than uniformly strong robustness. PGD-AT, Multi-AT, and AttackDRO++ show smaller and more balanced gap patterns, with AttackDRO++ avoiding the severe specialist inflation observed for DDN-AT.

6.3.3 Class-Level Transfer Patterns

We complement the aggregate analysis with a paired per-cell test. For each of the 80 attack–class cells, we compare the graybox accuracy of the AttackDRO++ target against each baseline target across the five training seeds (each cell pairs $n = 5$ seeds). A two-sided paired t-test is computed per cell.

Table 6.15: Per-(class $\times$ attack) graybox tests comparing AttackDRO++ against each baseline. Δ is computed as AttackDRO++ minus the baseline.

Baseline	Positive cells ($\Delta > 0$)	Sign-test p	Mean Δ (pp)	Significant cells AttackDRO++ / Baseline	Bottom-10 lift (pp)
Multi-AT	59/80	1.3×10^{-5}	$+0.557$	2 / 3	$+0.779$
PGD-AT	52/80	4.8×10^{-3}	$+2.332$	34 / 12	$+1.329$
DDN-AT	27/80	2.4×10^{-3}	-2.206	16 / 32	$+0.709$

Three observations follow. First, AttackDRO++ exceeds Multi-AT on 59 of 80 cells with a mean per-cell delta of $+0.557$ pp, in close agreement with the $+0.549$ pp aggregate from Table 6.11. Cell-level significance is sparse (2 vs. 3 cells reach $p < 0.05$ in either direction), as expected under the $n = 5$ per-cell sample size and the small absolute deltas; the bulk of the evidence for AttackDRO++ is therefore the *distributional* shift across many cells rather than a few cell-level landslides.

Second, the bottom-10 lift is positive in all three contrasts. The cells on which Multi-AT, PGD-AT, and DDN-AT are weakest concentrate on the cat, deer, dog, and bird classes under MI-FGSM, PGD-ℓ_∞, and PGD-ℓ_2. AttackDRO++ improves on Multi-AT by $+0.779$ pp on Multi-AT's worst 10 cells, and improves on PGD-AT by $+1.329$ pp on PGD-AT's worst 10 cells.

Third, the AttackDRO++ vs. DDN-AT cell-level comparison (-2.21 pp on average, 32 cells favoring DDN-AT at $p < 0.05$) is the per-cell counterpart of the DDN-AT graybox inflation: most of DDN-AT's wins are on ℓ_2 cells, while AttackDRO++ retains the wins it needs on the ℓ_∞ cells that DDN-AT collapses on under whitebox attack (Section 6.1.2).

The heatmaps in Figures 6.18–6.20 visualize these patterns explicitly: green cells favor AttackDRO++, red cells favor the baseline. The AttackDRO++ vs. Multi-AT heatmap is broadly green and small in magnitude; the AttackDRO++ vs. PGD-AT heatmap is broadly green with a few large red cells on ℓ_∞ attacks where PGD-AT specializes; and the AttackDRO++ vs. DDN-AT heatmap is mixed.

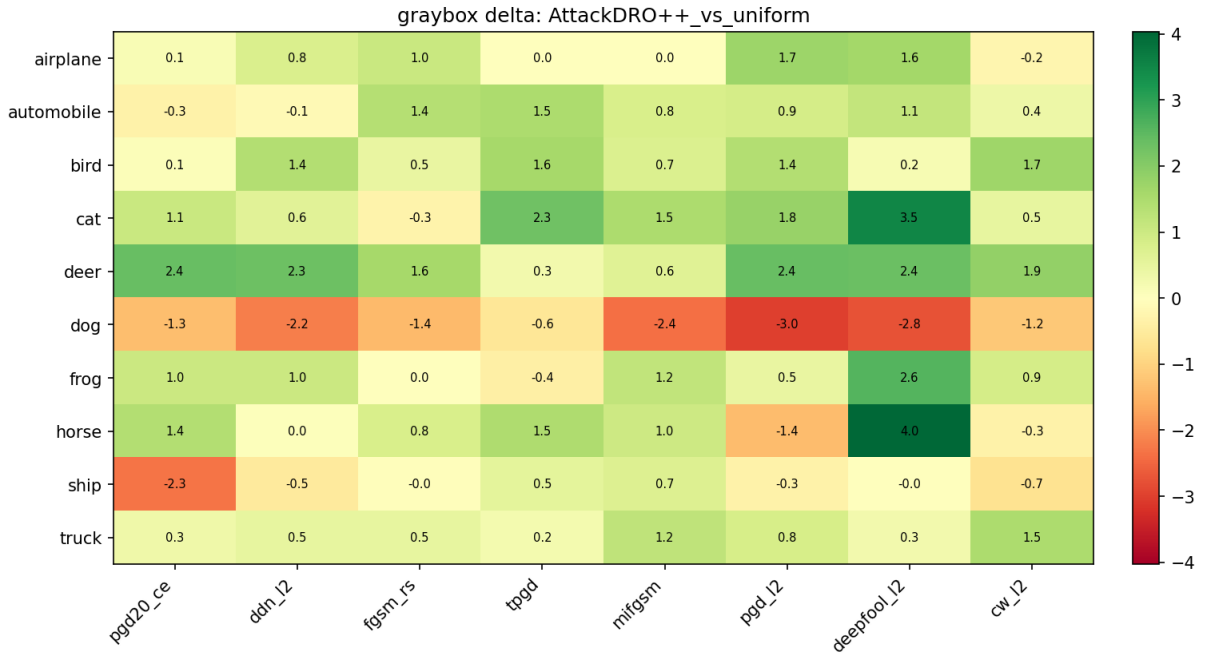


Figure 6.18: Per-(class $\times$ attack) graybox delta heatmap for AttackDRO++ (Ours) minus Multi-AT. Green favors AttackDRO++ (Ours); red favors Multi-AT.

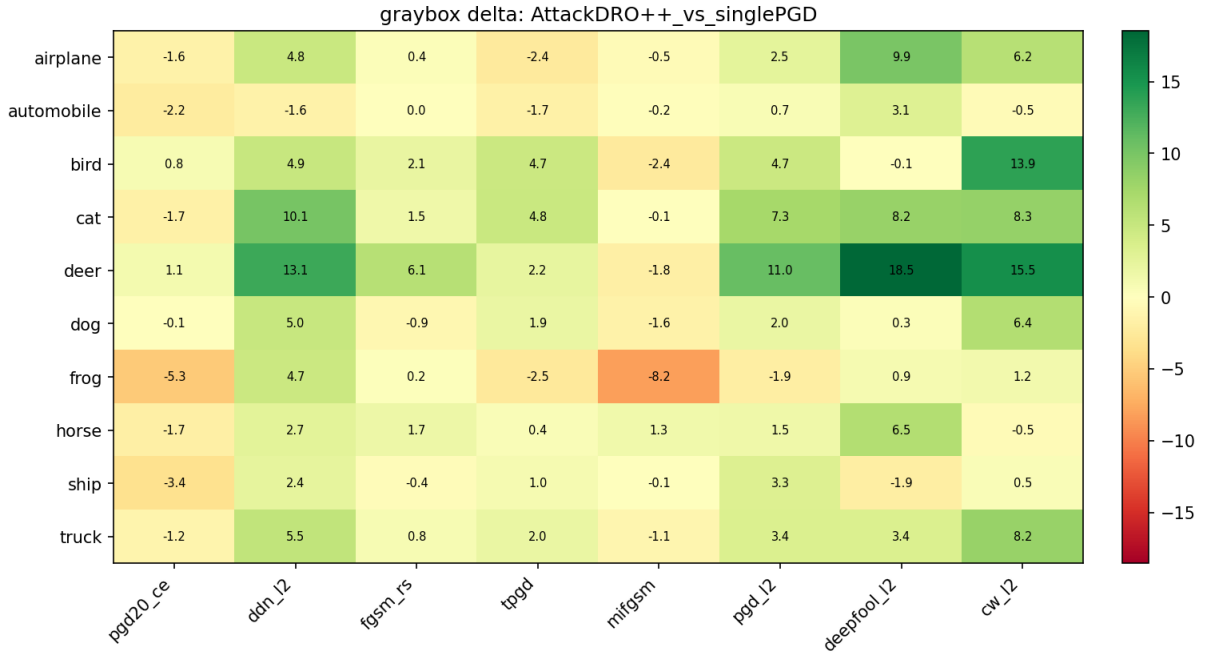


Figure 6.19: Per-(class × attack) graybox delta heatmap for AttackDRO++ (Ours) minus PGD-AT. Green favors AttackDRO++ (Ours); red favors PGD-AT.

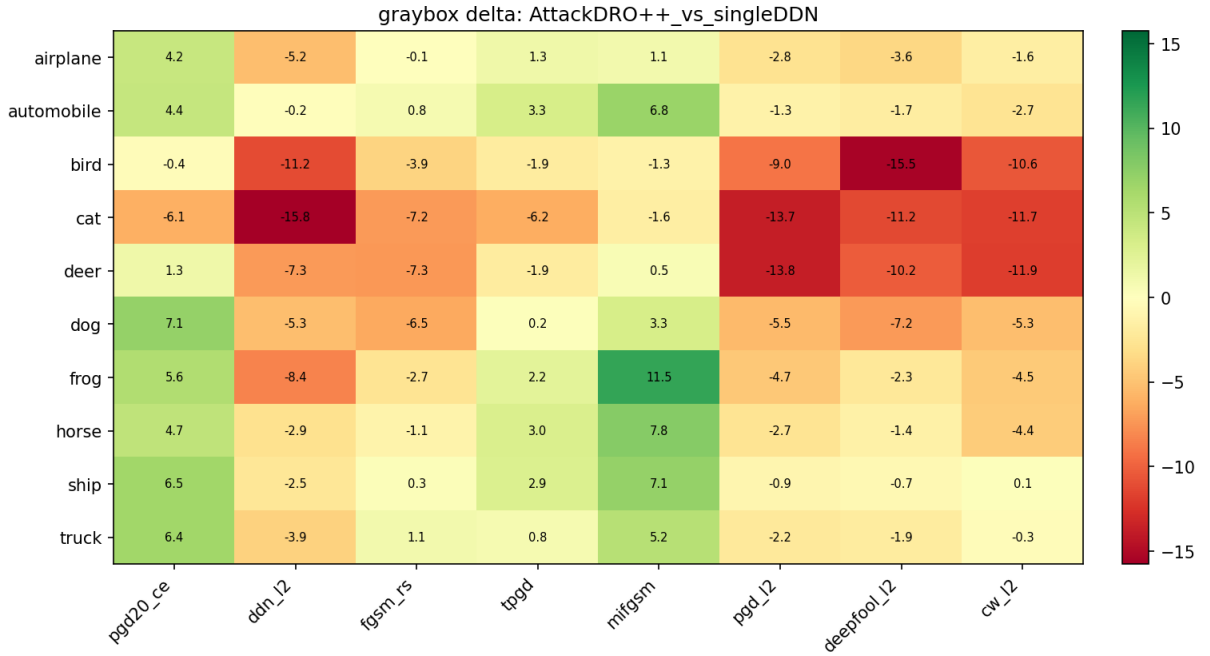


Figure 6.20: Per-(class × attack) graybox delta heatmap for AttackDRO++ (Ours) minus DDN-AT. Green favors AttackDRO++ (Ours); red favors DDN-AT.

Method contribution decomposition. The whitebox results of Section 6.1 compare AttackDRO++ against Multi-AT and against the two single-attack baselines as three separate contrasts. Under graybox transfer, we can additionally ask how much of the AttackDRO++ improvement over each single-attack baseline is contributed by multi-attack training (the Multi-AT component) versus the Cluster-DRO objective (the AttackDRO++ component on top of Multi-AT). We decompose the total AttackDRO++ vs. single-attack delta as

$$\Delta_{\text{total}} = \underbrace{(\text{Multi-AT}) - (\text{single-AT})}_{\Delta_{\text{multi-attack}}} + \underbrace{(\text{AttackDRO++}) - (\text{Multi-AT})}_{\Delta_{\text{Cluster-DRO}}},$$

and report this decomposition for whitebox Mean(8), cross-method graybox Mean(8), and the bottom-10-cell cross-method graybox average (the average graybox accuracy on the ten attack–class cells in which the target is weakest). The bottom-10 metric is included because it isolates worst-cell behavior, which is most sensitive to the AttackDRO++ objective.

Table 6.16: AttackDRO++ (Ours) gain decomposition. Values are percentage-point changes.

Metric	Base	Multi-AT	DRO++	Total
Whitebox Mean(8)	PGD-AT	$+0.934$	$+0.184$	$+1.118$
Whitebox Mean(8)	DDN-AT	$+5.536$	$+0.184$	$+5.720$
Cross-graybox Mean(8)	PGD-AT	$+1.775$	$+0.557$	$+2.332$
Cross-graybox Mean(8)	DDN-AT	-2.763	$+0.557$	-2.206
Bottom-10 graybox	PGD-AT	$+0.530$	$+0.799$	$+1.329$
Bottom-10 graybox	DDN-AT	-0.090	$+0.799$	$+0.709$

The Cluster-DRO effect (column 4) is positive on every metric and decomposition path, ranging from $+0.184$ pp on whitebox Mean(8) to $+0.799$ pp on the bottom-10 cross-graybox metric. Two implications follow. First, the Cluster-DRO effect is larger under graybox transfer than under direct whitebox attack, consistent with the $+0.279$ pp $\rightarrow$ $+0.549$ pp scaling observed in Section 6.3.2. Second, the Cluster-DRO effect is roughly four times larger on the bottom-10 cells ($+0.799$ pp) than on whitebox Mean(8), supporting the design intent that the Cluster-DRO objective primarily reshapes worst-cell behavior rather than the average.

The multi-attack column tells a complementary specialist story. Going from single-AT

to Multi-AT adds $+5.54$ pp of whitebox Mean(8) when the baseline is DDN-AT, but removes -2.76 pp of cross-graybox Mean(8) for the same baseline — because DDN-AT's graybox accuracy is inflated by the very ℓ_∞ transfer mismatch that Multi-AT corrects. The cluster-DRO component remains positive in both cases, indicating that it adds robustness that is not absorbed by either multi-attack training alone or DDN-AT's structural graybox advantage.

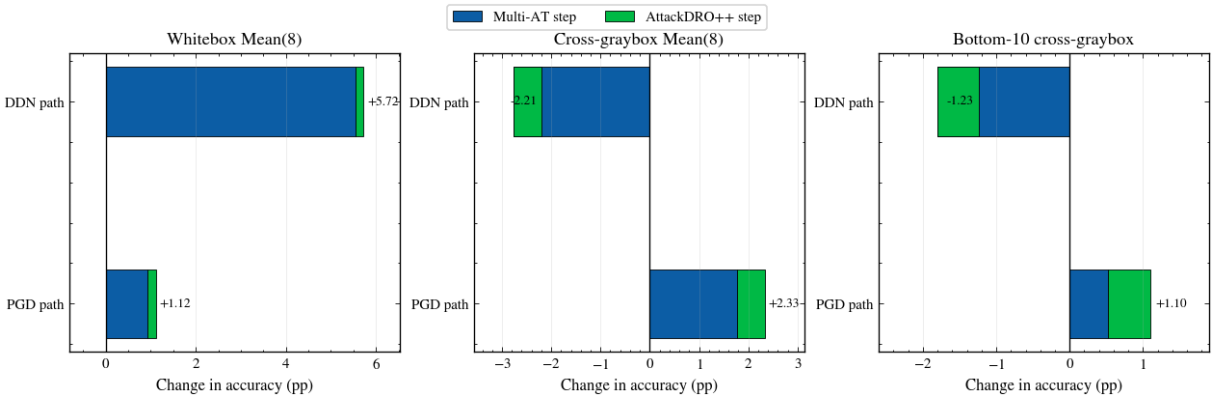


Figure 6.21: Visual decomposition of AttackDRO++ improvement over the single-attack baselines into a multi-attack effect (Multi-AT minus single-AT) and a Cluster-DRO effect (AttackDRO++ minus Multi-AT) across whitebox, cross-method graybox, and bottom-10 cross-method graybox metrics, for the two single-AT decomposition paths.

The graybox analysis tests whether the whitebox pattern survives transfer from independently trained surrogate checkpoints. It supports the same broad interpretation as the direct whitebox evaluation, but transfer matrices and paired comparisons do not identify which design choices make AttackDRO++ stable. The next section therefore turns to hyperparameter sensitivity, focusing on cluster granularity, anchor strength, and refresh frequency.

6.4 Sensitivity to Key Hyperparameters

The preceding sections evaluated the final AttackDRO++ configuration under whitebox, class-wise, and graybox settings. Those results show how the method behaves once the design choices are fixed, but they do not explain how sensitive the method is to those choices. This section therefore studies three practical hyperparameters: the number of discovered clusters, the strength of the uniform anchor, and the frequency with which clusters are refreshed.

These ablations are intended as configuration guidance rather than as a second main comparison. Unless otherwise stated, they use the same CIFAR-10, ResNet-18, source-attack, and evaluation-attack protocol defined in Chapter 5. Mean(8) and Worst(8) follow the

definitions in Section 5.7: both are computed over the eight non-AutoAttack evaluation attacks, while AutoAttack-ℓ_∞ is reported separately.

6.4.1 Number of Clusters: How Many Groups Are Needed?

The number of clusters controls the granularity of the latent groups used by AttackDRO++. If K is too small, distinct failure modes may be merged into the same group, making the Cluster-DRO correction too coarse. If K is too large, the minibatch may be split into many small groups, making group-loss estimates noisier and the cluster weights less stable.

Table 6.17 summarizes the cluster-count ablation. Across the available runs, all settings remain close in Mean(8), with differences well below one percentage point. The default $K = 4$ setting gives the highest Worst(8) among the tested settings and remains competitive in Mean(8), while $K = 2$ is slightly more coarse and $K = 6$ or $K = 10$ do not provide a clear gain. The very fine $K = 10$ setting should be interpreted cautiously because only two valid $K = 10$ runs are available in the current log.

Table 6.17: Number-of-clusters ablation for AttackDRO++ with anchor strength $\lambda_{\text{DRO}} = 0.35$. Values are robust accuracy (%) reported as mean $\pm$ standard deviation across available runs. Mean(8) and Worst(8) are computed over the eight non-AutoAttack attacks; AutoAttack-ℓ_∞ is reported separately. Bold values indicate the best setting for each metric.

Setting	K	Runs	Mean(8) ↑	Worst(8) ↑	AutoAttack ↑
Coarse grouping	2	3	62.33 ± 0.06	45.13 ± 0.27	40.95 ± 0.90
Default grouping	4	3	$\mathbf{62.35 \pm 0.20}$	$\mathbf{45.33 \pm 0.25}$	$\mathbf{41.14 \pm 1.15}$
Finer grouping	6	3	62.28 ± 0.14	45.17 ± 0.47	40.95 ± 1.30
Very fine grouping	10	2	62.35 ± 0.09	45.17 ± 0.08	40.62 ± 1.11

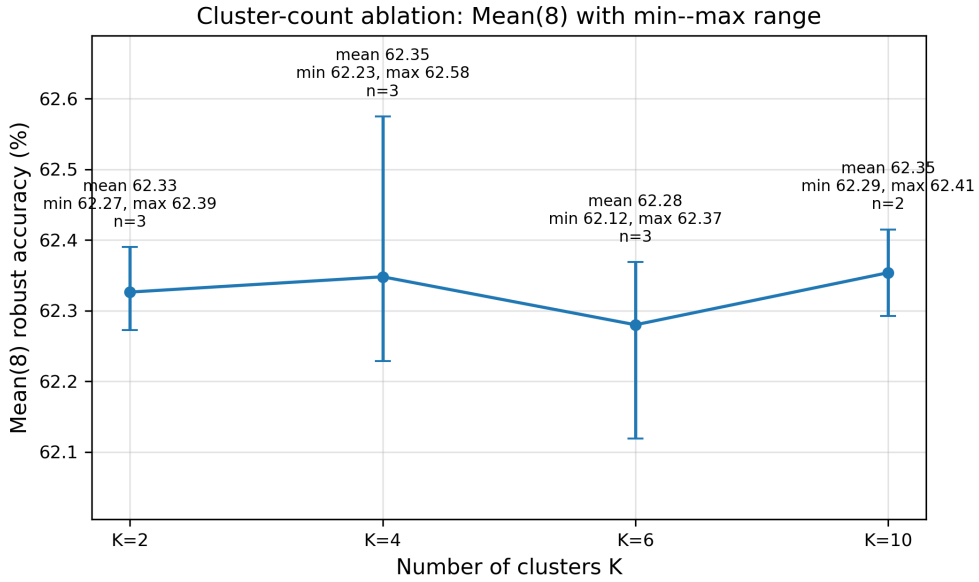


Figure 6.22: Sensitivity of AttackDRO++ to the number of clusters. Mean(8), Worst(8), and AutoAttack-ℓ_∞ are reported in percent. Mean(8) and Worst(8) are computed over the eight non-AutoAttack attacks; AutoAttack-ℓ_∞ is reported separately.

The ablation supports using a moderate number of clusters rather than treating the cluster count as a highly tuned parameter. The default $K = 4$ is expressive enough to go beyond the two source attack identities, but it does not fragment the training batch as aggressively as larger values. Therefore, the evidence supports $K = 4$ as a stable practical choice, not as a universally optimal cluster count.

6.4.2 Anchor Strength: Finding a Stable Trade-off

The anchor strength λ_{DRO} controls the interpolation between the uniform multi-attack ERM objective and the adaptive Cluster-DRO objective. A smaller value keeps training closer to the uniform baseline, while a larger value gives the cluster-reweighted loss more influence. In this ablation, the previous $\lambda_{\text{DRO}} = 1.0$ setting is removed because it corresponds to an overly aggressive Cluster-DRO setting and is not part of the final candidate set.

Table 6.18 compares three anchor strengths. The differences are again small, but the moderate setting $\lambda_{\text{DRO}} = 0.35$ gives the strongest Mean(8), Worst(8), and AutoAttack values among the tested settings. The lower value $\lambda_{\text{DRO}} = 0.20$ remains close, suggesting that the method does not require a sharply tuned anchor. The higher value $\lambda_{\text{DRO}} = 0.50$ slightly reduces Mean(8) and AutoAttack, which is consistent with the idea that too much emphasis on adaptive cluster weighting can reduce the stabilizing effect of the uniform multi-attack baseline.

Table 6.18: Anchor-strength ablation for AttackDRO++ with $K = 4$. Values are robust accuracy (%) reported as mean $\pm$ standard deviation across three runs. Mean(8) and Worst(8) are computed over the eight non-AutoAttack attacks; AutoAttack-ℓ_∞ is reported separately. Bold values indicate the best setting for each metric.

Anchor strength λ_{DRO}	Runs	Mean(8) ↑	Worst(8) ↑	AutoAttack ↑
0.20	3	62.28 ± 0.09	45.01 ± 0.40	41.08 ± 1.66
0.35 (default)	3	$\mathbf{62.35 \pm 0.20}$	$\mathbf{45.33 \pm 0.25}$	$\mathbf{41.14 \pm 1.15}$
0.50	3	62.24 ± 0.18	45.24 ± 0.40	40.24 ± 1.09

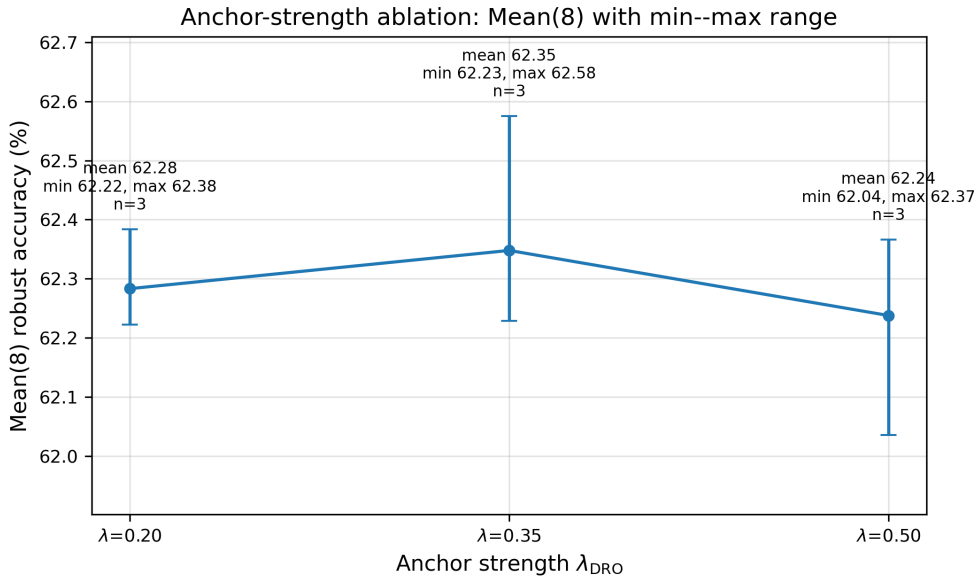


Figure 6.23: Sensitivity of AttackDRO++ to the anchor strength λ_{DRO}. Mean(8), Worst(8), and AutoAttack-ℓ_∞ are reported in percent. The previous $\lambda_{\text{DRO}} = 1.0$ setting is excluded from the final ablation panel.

Overall, the anchor-strength ablation supports the design motivation from Chapter 4. AttackDRO++ benefits from a nonzero cluster-level correction, but the correction should remain anchored to the uniform multi-attack objective. The default value $\lambda_{\text{DRO}} = 0.35$ is retained because it gives the best balance among the tested settings without relying entirely on either uniform ERM or aggressive Cluster-DRO weighting.

6.4.3 Recluster Frequency: How Often Should Groups Update?

The cluster-refresh interval R controls how often the K-means clusterer is refitted from the feature bank. Refreshing every epoch may adapt faster to changes in the representation, but it also changes the group structure more frequently. Refreshing less often may give a more stable grouping signal, but it may lag behind the evolving model.

Table 6.19 compares refreshing every one epoch with refreshing every two epochs. The two schedules produce nearly identical Mean(8), Worst(8), and family-level behavior. The default two-epoch schedule gives a slightly higher Mean(8) and ℓ_∞-family average, while the one-epoch schedule gives a slightly higher Worst(8), AutoAttack, and ℓ_2-family average. These differences are small relative to the three-run setting, so the ablation does not show a strong advantage for either schedule.

Table 6.19: Cluster-refresh schedule ablation for AttackDRO++ with $\lambda_{\text{DRO}} = 0.35$, gradient fingerprints, and $K = 4$. Values are robust accuracy (%) reported as mean $\pm$ standard deviation across three runs. Mean(8) and Worst(8) are computed over the eight non-AutoAttack attacks; AutoAttack-ℓ_∞ is reported separately. Bold values indicate the best setting for each metric.

Refresh interval	Runs	Mean(8) ↑	Worst(8) ↑	AutoAttack ↑	ℓ_2-family avg. ↑	ℓ_∞-family avg. ↑
1 epoch	3	62.38 ± 0.14	$\mathbf{45.17 \pm 0.34}$	$\mathbf{40.30 \pm 1.37}$	$\mathbf{67.57 \pm 0.16}$	57.18 ± 0.27
2 epochs (default)	3	$\mathbf{62.44 \pm 0.22}$	45.14 ± 0.23	39.45 ± 0.68	67.56 ± 0.08	$\mathbf{57.31 \pm 0.35}$

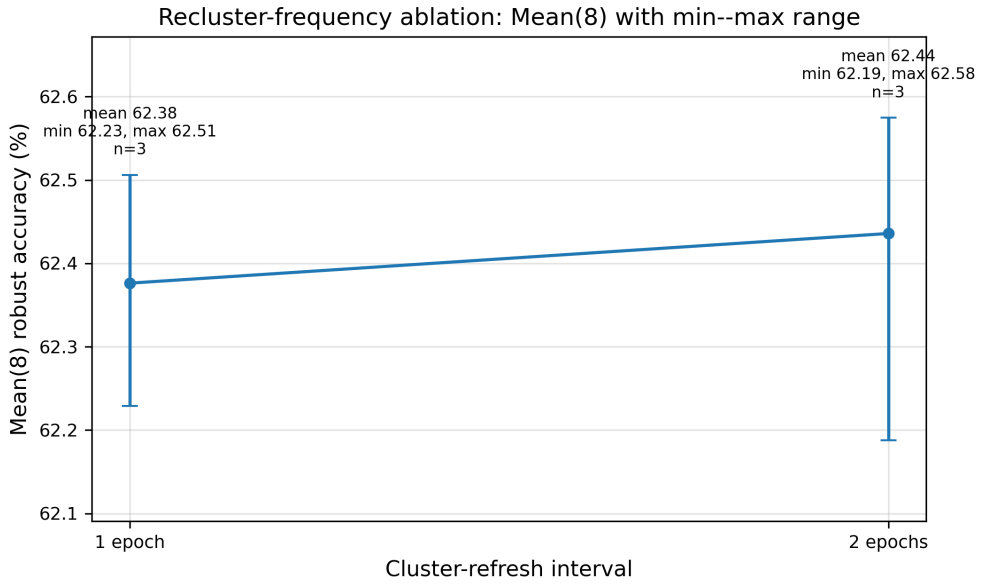


Figure 6.24: Sensitivity of AttackDRO++ to the cluster-refresh interval. Mean(8), Worst(8), and AutoAttack-ℓ_∞ are reported in percent. The two refresh schedules produce similar results, so the default two-epoch schedule is retained as a practical stability choice.

We retain $R = 2$ epochs as the default because it gives comparable robustness while reducing the frequency of cluster refitting. This is a practical choice: the ablation suggests that the method is not highly sensitive to the exact refresh interval, so the less frequent schedule is preferred for stability and simplicity.

Supplementary architecture check. A single-run WRN-28-10 check is reported in Appendix C as supplementary evidence. This check is not included in the main paired claims because it does not use the repeated-random-seed protocol of the main ResNet-18 experiments. In that setting, Multi-AT slightly outperforms AttackDRO++ on both Mean(8) and AutoAttack-ℓ_∞, suggesting that the relative benefit of the proposed grouping strategy may depend on model capacity and should be evaluated more systematically in future work.

Together, these ablations suggest that the final AttackDRO++ configuration is not the result of a fragile hyperparameter choice. Moderate clustering granularity, moderate anchor strength, and a two-epoch refresh schedule provide a stable practical configuration. The ablations also define the limits of the current evidence: they support the selected ResNet-18 configuration, but broader architecture-level conclusions require additional repeated-seed experiments. Chapter 6 evaluated AttackDRO++ through four connected evidence panels. The aggregate whitebox results first showed that the proposed method improves average and held-out robustness relative to Multi-AT while leaving the strongest standardized checks broadly unchanged. The per-attack and class-wise drill-downs then clarified where this behavior comes from: the method acts less like a single-norm specialist and more like a balanced multi-attack model, although hard animal-class cells under strong ℓ_∞ attacks remain persistent failure cases.

The graybox analysis tested whether the same pattern survives transferred attacks from independently trained surrogate checkpoints. It supported the whitebox interpretation while also showing that transfer accuracy must be read carefully because specialist models can appear strong when attacks transfer poorly across threat geometries. Finally, the ablation section examined whether the observed behavior depends on key design choices, treating cluster count, anchor strength, and refresh frequency as sensitivity checks rather than standalone proof.

Overall, the supported conclusion is conservative: within the CIFAR-10 and ResNet-18 setting studied here, AttackDRO++ improves cross-attack balance over the uniform multi-attack baseline without establishing universal dominance over single-attack specialists. Chapter 7 summarizes these findings, states the limitations, and outlines future extensions.

Chapter 7

Conclusion and Future Work

7.1 Summary of Contributions

7.2 Answers to Research Questions

7.3 Limitations of the Current Study

7.4 Directions for Future Work

7.5 Closing Remarks

Bibliography

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.

[2] J. Rony, L. G. Hafemann, L. S. Oliveira, I. B. Ayed, R. Sabourin, and E. Granger, "Decoupling direction and norm for efficient gradient-based l2 adversarial attacks and defenses," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 4322–4330.

[3] F. Croce and M. Hein, "Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks," in *International Conference on Machine Learning*, 2020, pp. 2206–2216.

Appendix A

Illustrative Pseudo-code and Flowcharts

Appendix B

Experimental Logs and Configurations

Appendix C

Additional Tables and Visualizations

Table C.1: Supplementary WRN-28-10 architecture check on CIFAR-10. Values are robust accuracy (%) from a single run and are therefore not included in the main paired claims. Mean(8) and Worst(8) are computed over the eight non-AutoAttack attacks; AutoAttack-ℓ_∞ is reported separately. Bold values indicate the best method for each metric.

Method	Mean(8) ↑	Worst(8) ↑	AutoAttack ↑
Multi-AT	**64.87**	**47.99**	**44.92**
AttackDRO++ (Ours)	64.68	47.72	43.16

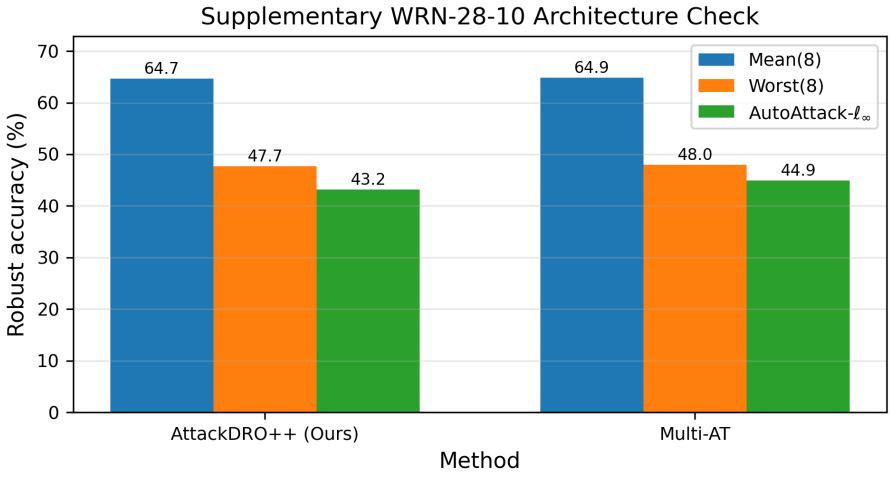


Figure C.1: Supplementary WRN-28-10 architecture check on CIFAR-10. Values are from a single run and are therefore not included in the main paired claims. Mean(8) and Worst(8) are computed over the eight non-AutoAttack attacks; AutoAttack-ℓ_∞ is reported separately.

Appendix D

Compute Resources and Environment Notes